COSC 2633/2637
Big Data Processing

Lecture 3 - MapReduce Basics

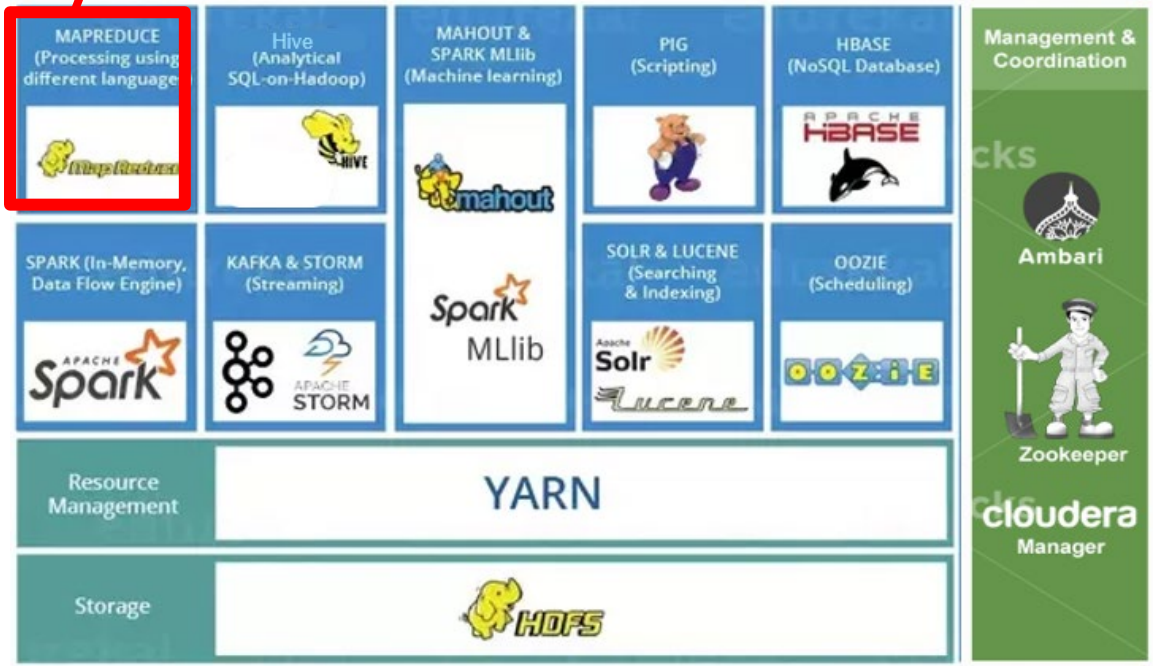
RMIT UNIVERSITY

Part 1

Typical Big Data Problem

Apache Hadoop Ecosystem

- **Hadoop MapReduce**
  - A programming model for large-scale data processing.



Typical Big Data Problem

- Iterate over a large number of records
- Extract something of interest from each

Map

- Shuffle and sort intermediate results
- Aggregate intermediate results
- Generate final output

Reduce

(Dean and Ghemawat, OSDI 2004)

Power of MapReduce

Simplicity - MapReduce programmer

- Preparing the input data
- Implement mapper, reducer
- Optionally implement combiner and partitioner

MapReduce

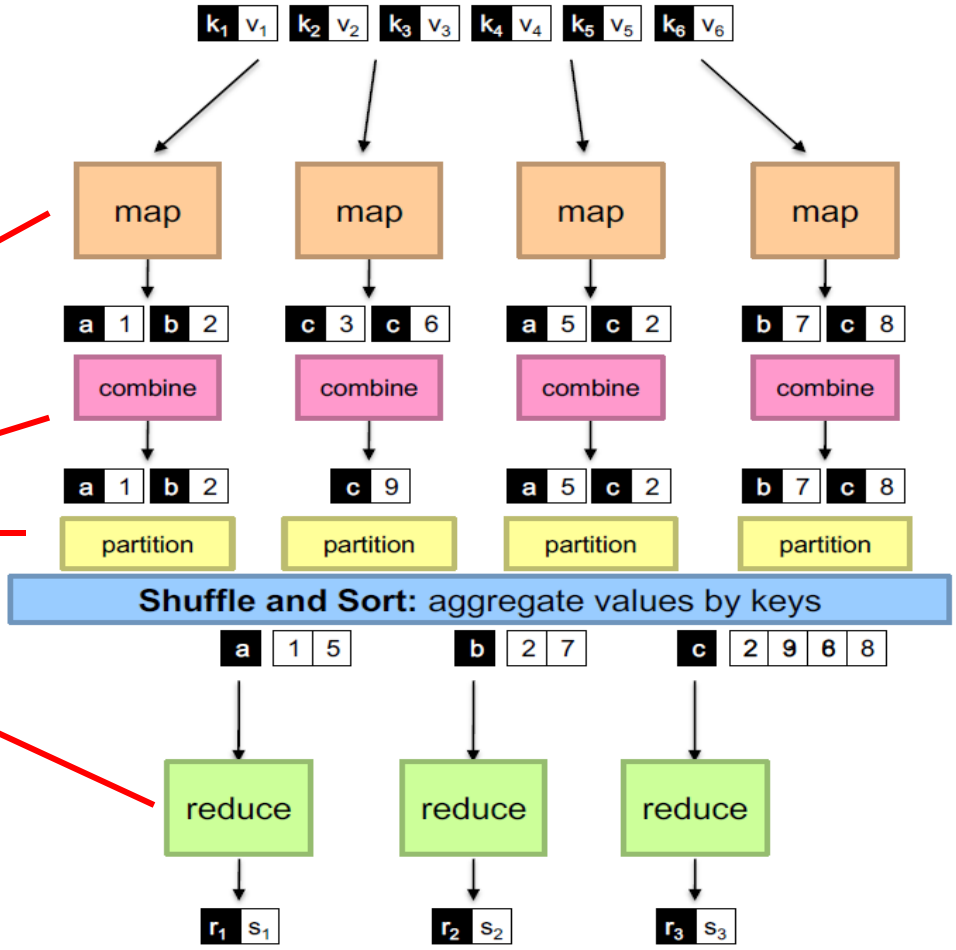
Programmers just specify four functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

combine $(k_2, [v_2]) \rightarrow [<k_2, v_2>]$

partition (k', number of partitions) → partition for k'

reduce (k2, [v2]) → [<k3, v3>]



MapReduce

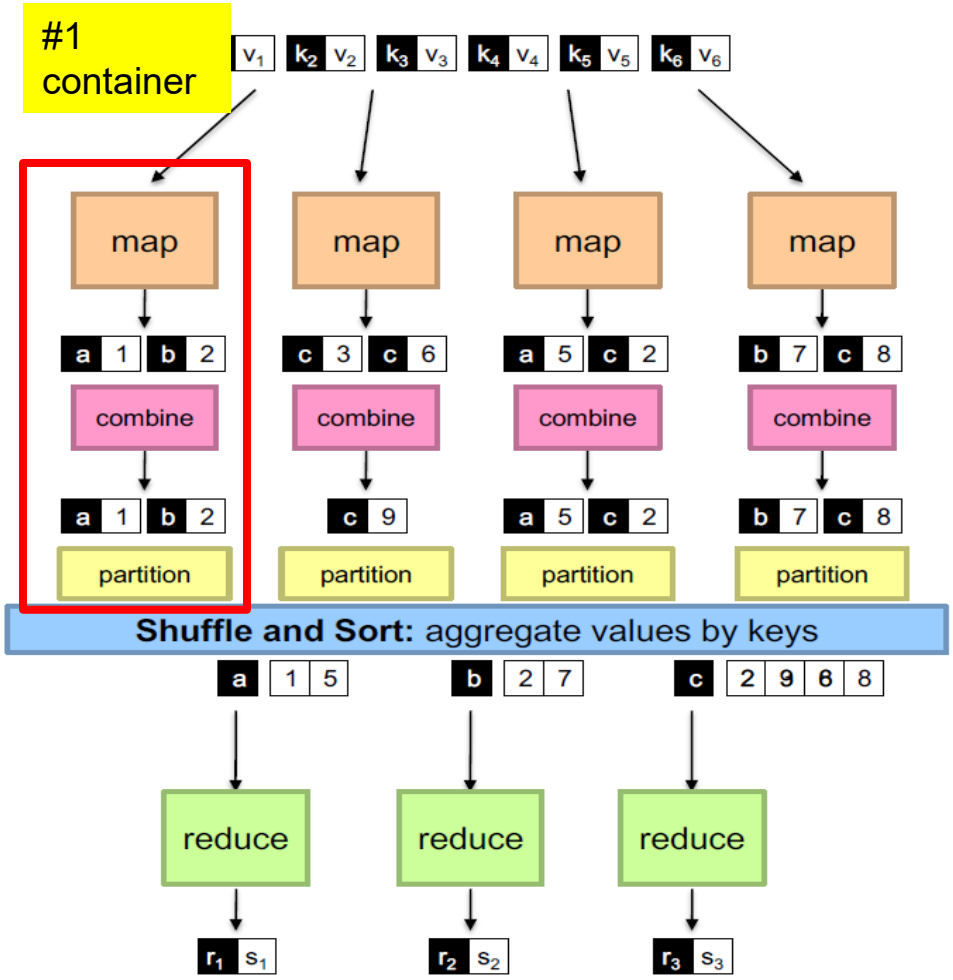
Programmers just specify four functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

combine $(k_2, [v_2]) \rightarrow [<k_2, v_2>]$

partition (k', number of partitions) → partition for k'

reduce (k2, [v2]) → [<k3, v3>]



MapReduce

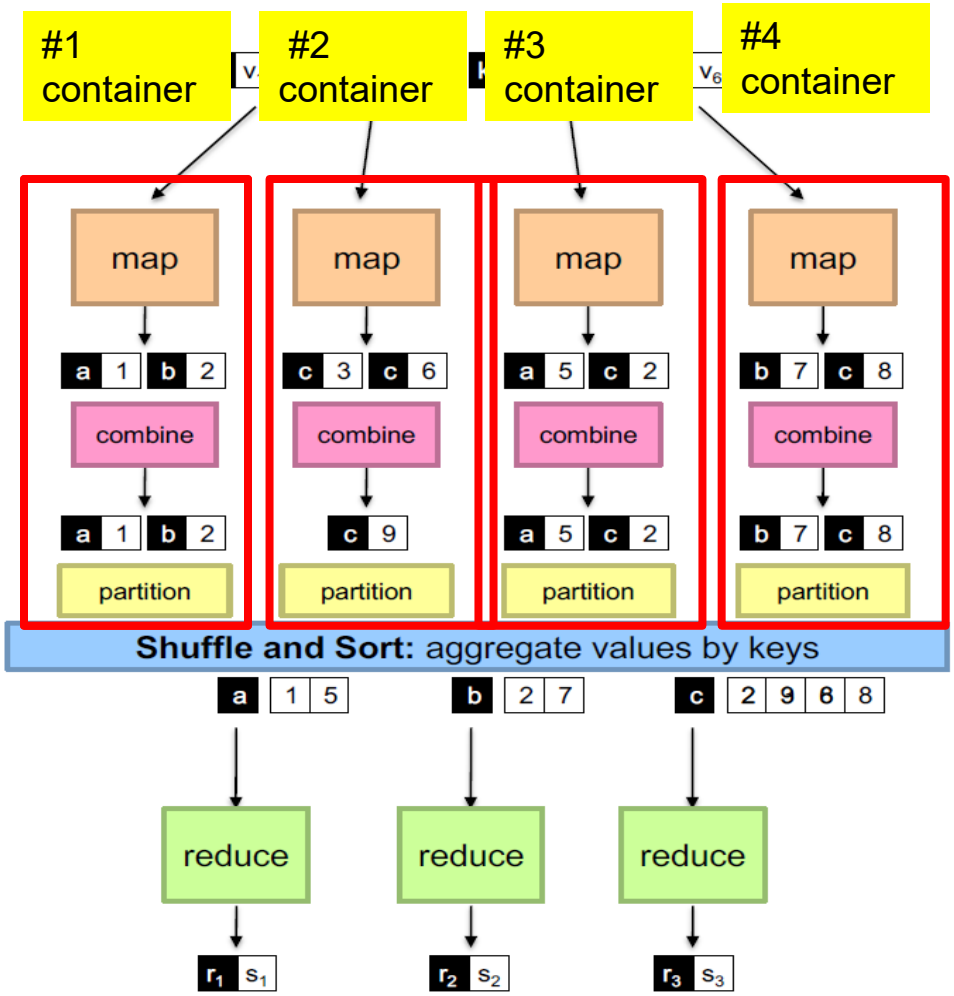
Programmers just specify four functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

combine $(k_2, [v_2]) \rightarrow [<k_2, v_2>]$

partition (k', number of partitions) → partition for k'

reduce (k2, [v2]) → [<k3, v3>]



MapReduce

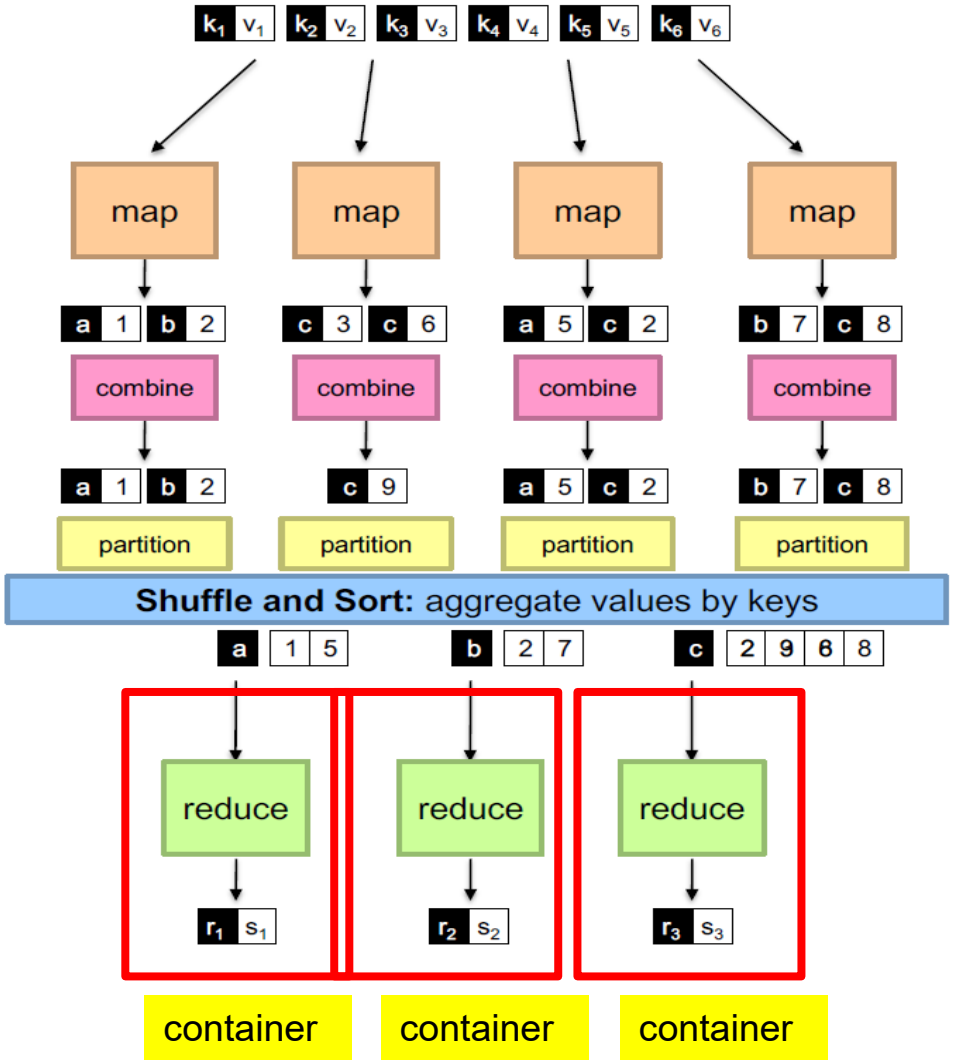
Programmers just specify four functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

combine $(k_2, [v_2]) \rightarrow [<k_2, v_2>]$

partition (k', number of partitions) → partition for k'

reduce (k2, [v2]) → [<k3, v3>]



MapReduce

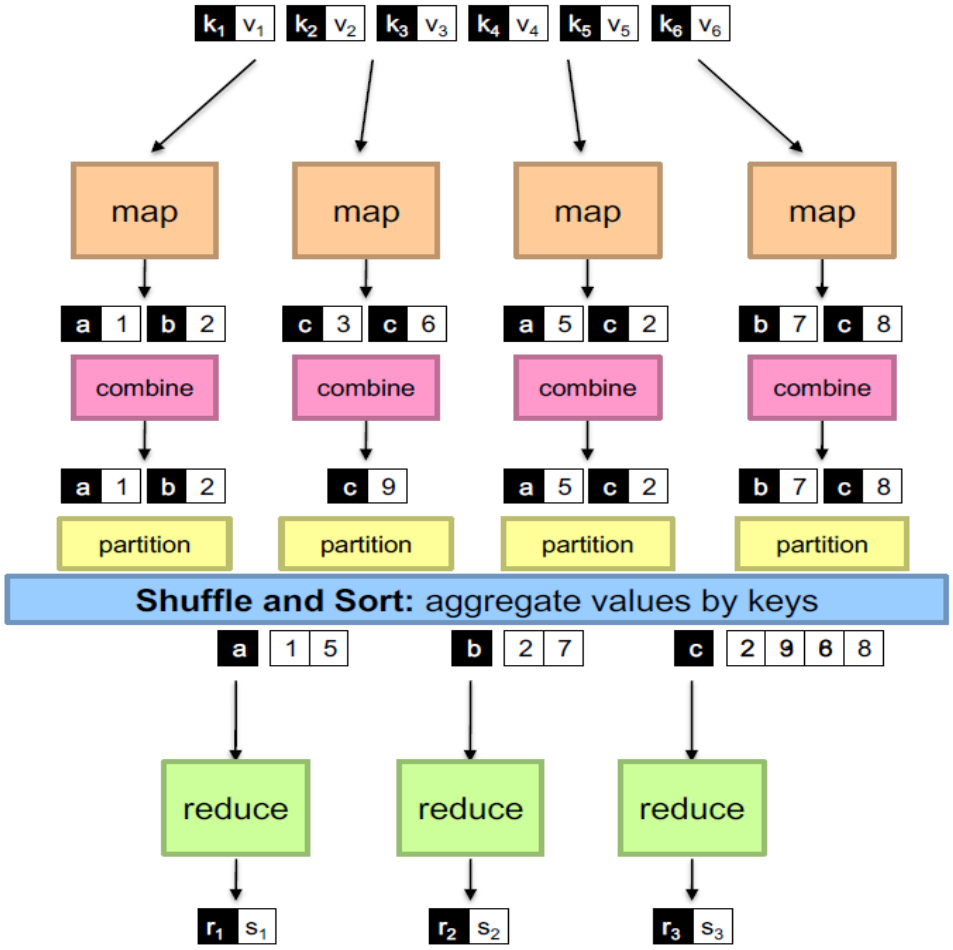
Programmers just specify four functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

combine $(k_2, [v_2]) \rightarrow [<k_2, v_2>]$

partition (k', number of partitions) → partition for k'

reduce (k2, [v2]) → [<k3, v3>]



MapReduce Execution Framework transparently handles everything else…

End of Part 1

Part 2

MapReduce Execution Platform

MapReduce Execution Framework transparently handles everything else…

- **A key idea is to separate "*what*" from "*how*"**
  - Developer specifies the computation that needs to be performed
  - Execution framework (also called "*runtime*" sometimes) handles actual execution

What are included?

MapReduce Execution Framework transparently handles everything else…

- Handles scheduling
  - Assigns workers to map and reduce tasks
- Handles "data distribution"
  - Moves processes to data
- Handles synchronization
  - Gathers, sorts, and shuffles intermediate data
- Handles errors and faults
  - Detects worker failures and restarts

Handles scheduling

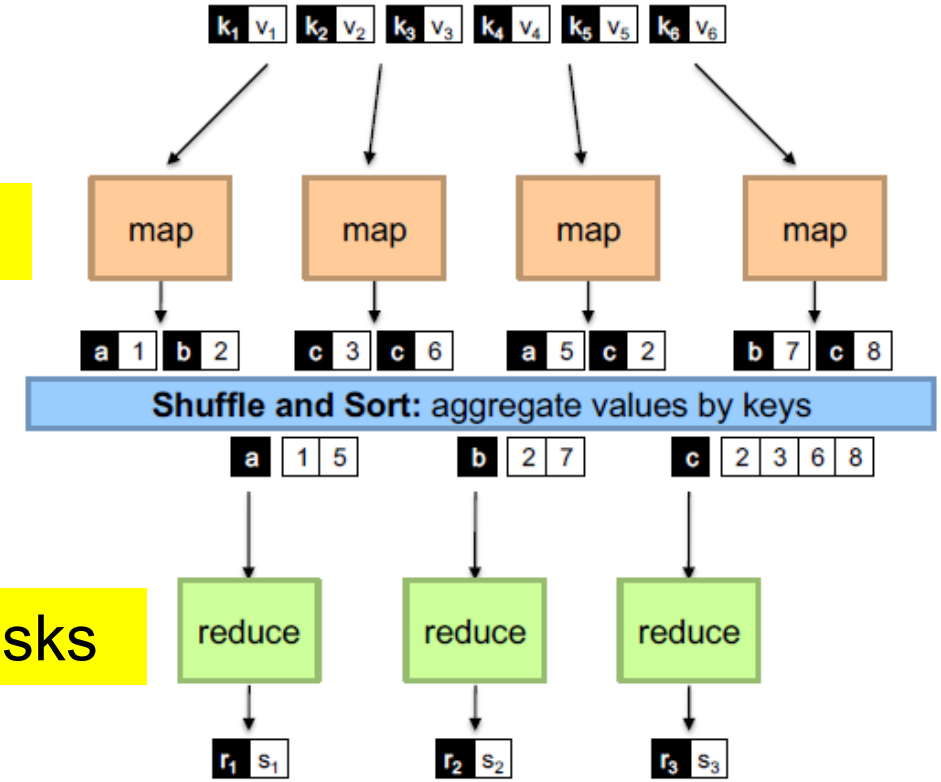
Each MapReduce job is divided into smaller units called tasks.

Handles scheduling

Each MapReduce job is divided into smaller units called tasks.

e.g., 4 smaller map tasks

e.g., 3 smaller reduce tasks



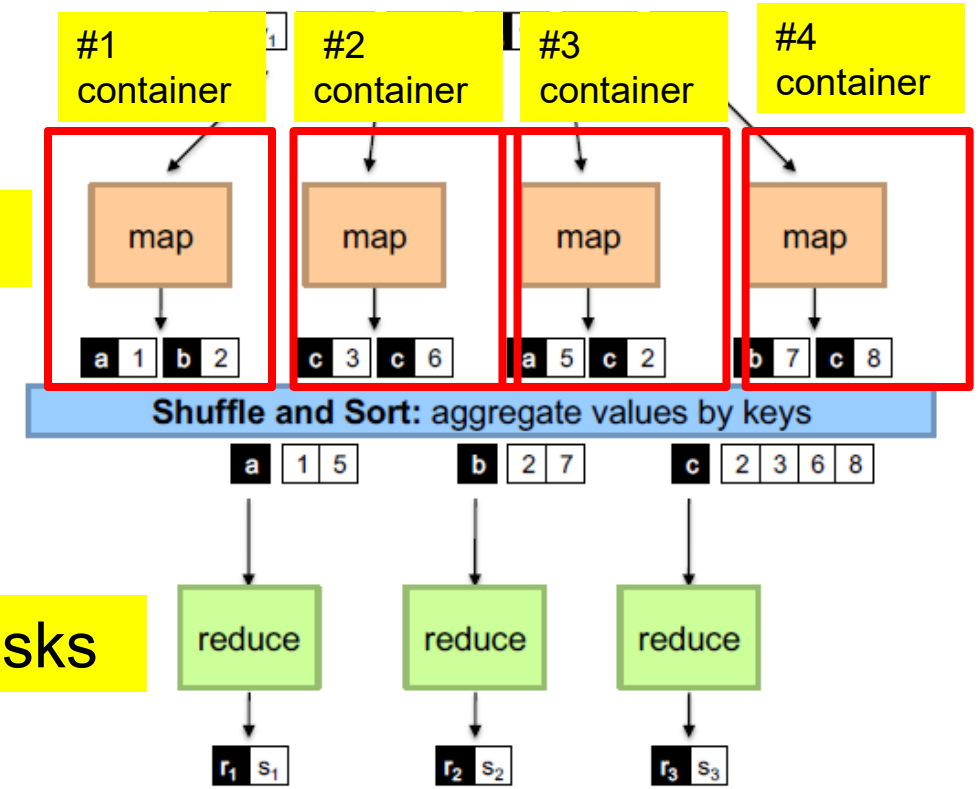
Handles scheduling

Each MapReduce job is divided into smaller units called tasks.

e.g., 4 smaller map tasks

Assign workers (containers) to map and reduce tasks

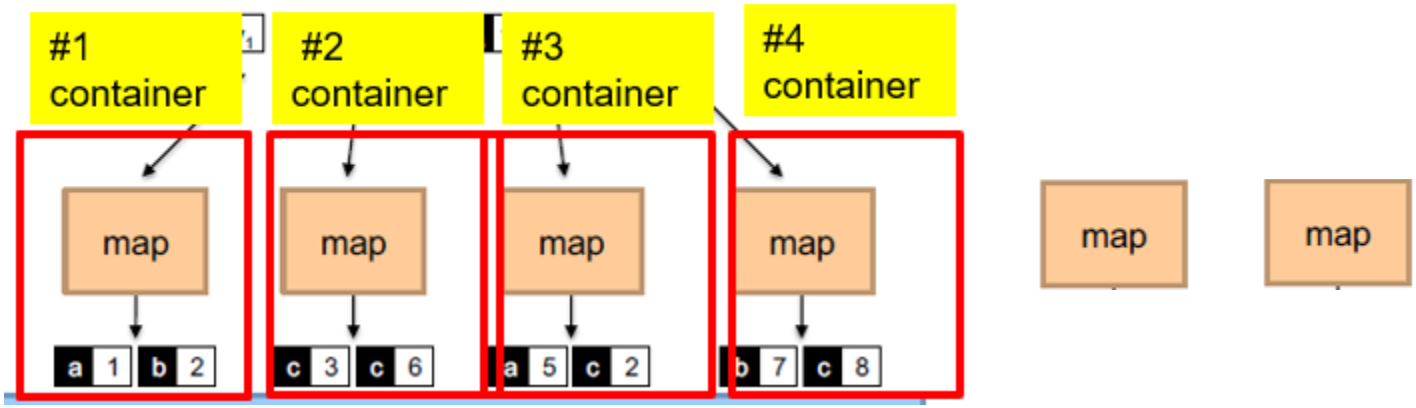
e.g., 3 smaller reduce tasks



Handles scheduling

Each MapReduce job is divided into smaller units called tasks.

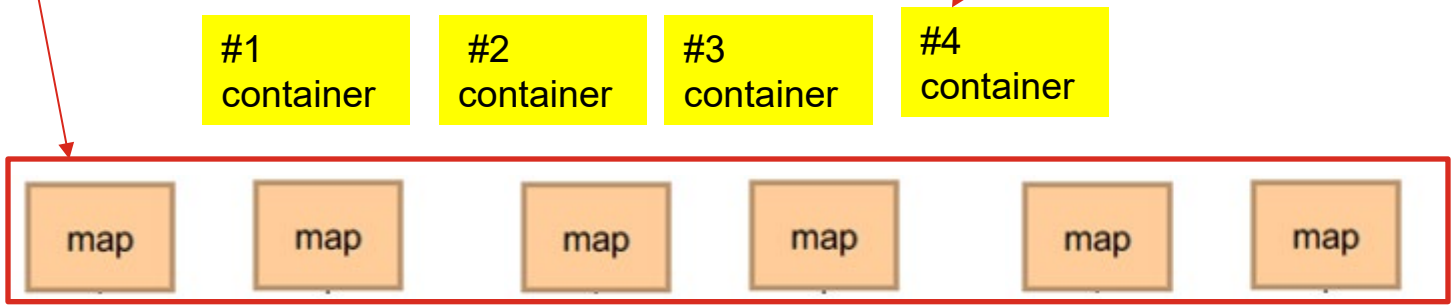
- For a large number of jobs, the total number of tasks may exceed the number of tasks that can be run on the cluster concurrently.



Handles scheduling

Each MapReduce job is divided into smaller units called tasks.

- For a large number of jobs, the total number of tasks may exceed the number of tasks that can be run on the cluster concurrently.
- A task queue needs to be maintained and scheduling which task to be processed once computing resource is available.



Handles scheduling

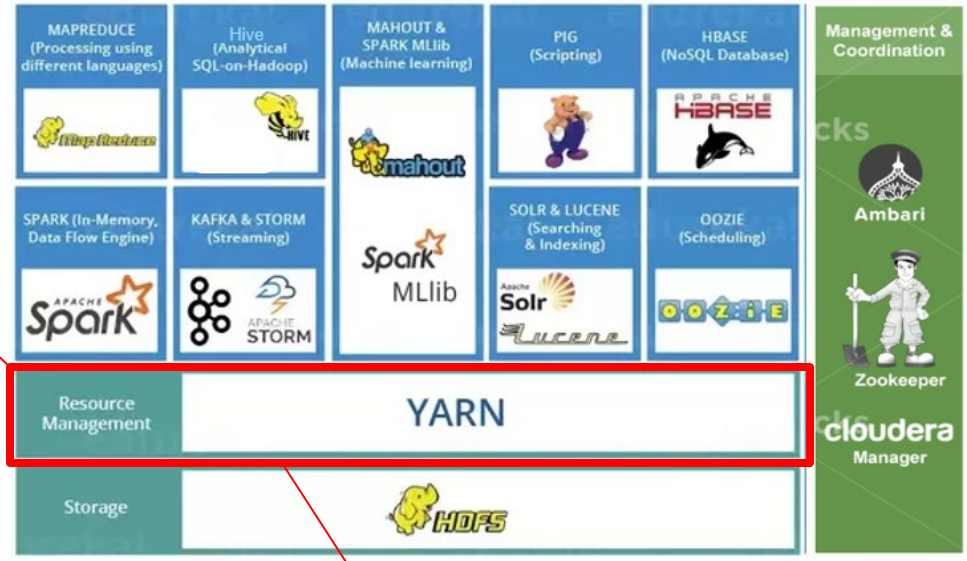
Each MapReduce job is divided into smaller units called tasks.

- For a large number of jobs, the total number of tasks may exceed the number of tasks that can be run on the cluster concurrently.
- A task queue needs to be maintained and scheduling which task to be processed once a cluster node is available.

Among different jobs, scheduling involves coordination of their tasks.

Handles scheduling – Yarn

The ResourceManager (RM) in Yarn tracks resources on a cluster, and assigns them to applications (jobs) that need them.

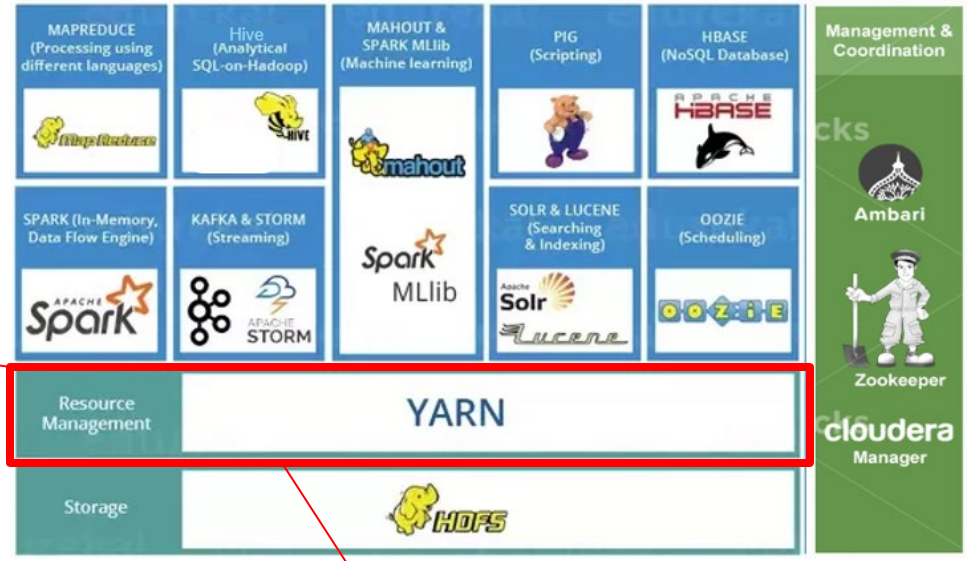


Handles scheduling

Handles scheduling – Yarn

The ResourceManager (RM) in Yarn tracks resources on a cluster, and assigns them to applications (jobs) that need them.

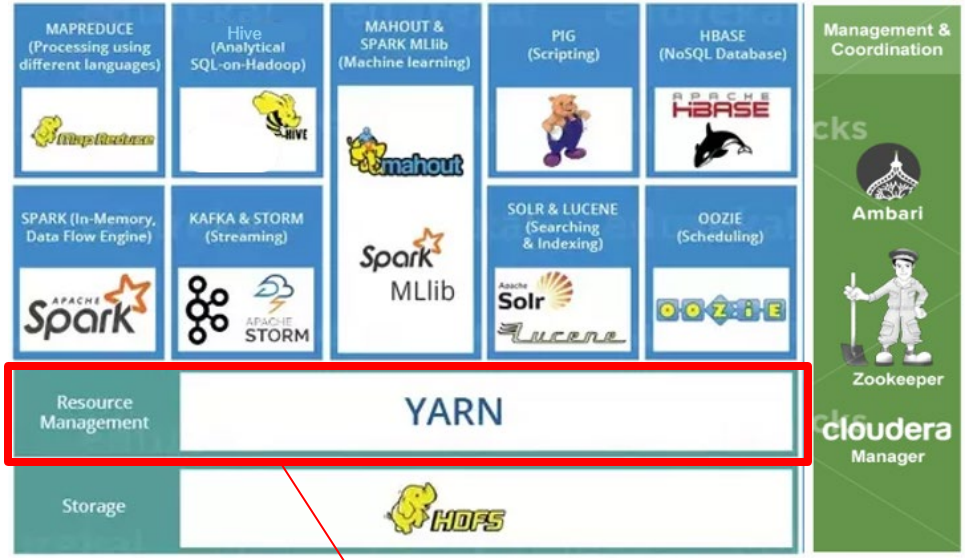
- The scheduler is that part of the RM that manages computing resource sharing in a cluster among many tasks from same/different jobs.



Handles scheduling – Yarn

The ResourceManager (RM) in Yarn tracks resources on a cluster, and assigns them to applications that need them.

- The scheduler is that part of the RM that manages computing resource sharing in a cluster among many tasks from same/different jobs.



Handles scheduling

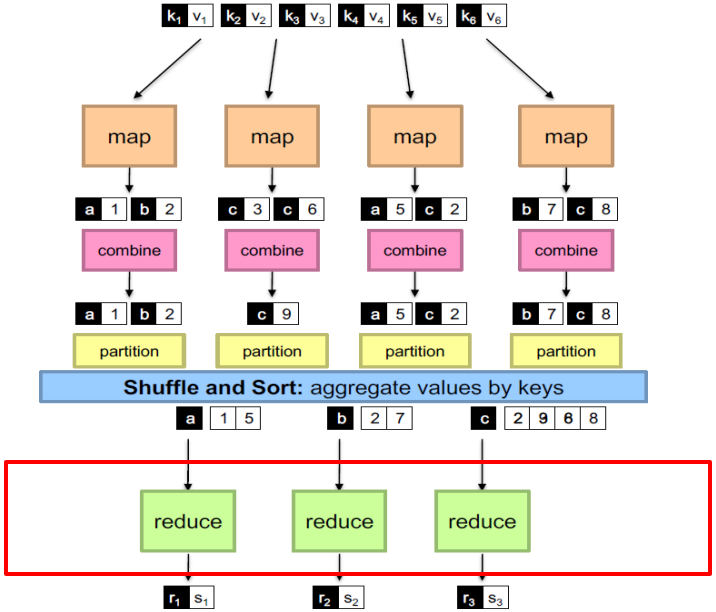
Fair Scheduler

- The Fair Scheduler is a popular choice (recommended by Cloudera) among the schedulers YARN supports.
- In its simplest form, it shares resources fairly among all jobs running on the cluster.

Handles scheduling – Straggler

The speed of MapReduce job is bounded by the running time of the slowest reduce task.

- Straggler: the task taken an unusually long time to complete.



Handles scheduling – Straggler

Straggler - Two Causes

- Machine suffering from broken hardware
  - Solution: speculative execution (an identical copy of the same task is executed on different machines and uses the result of the first task complete.)

Animation

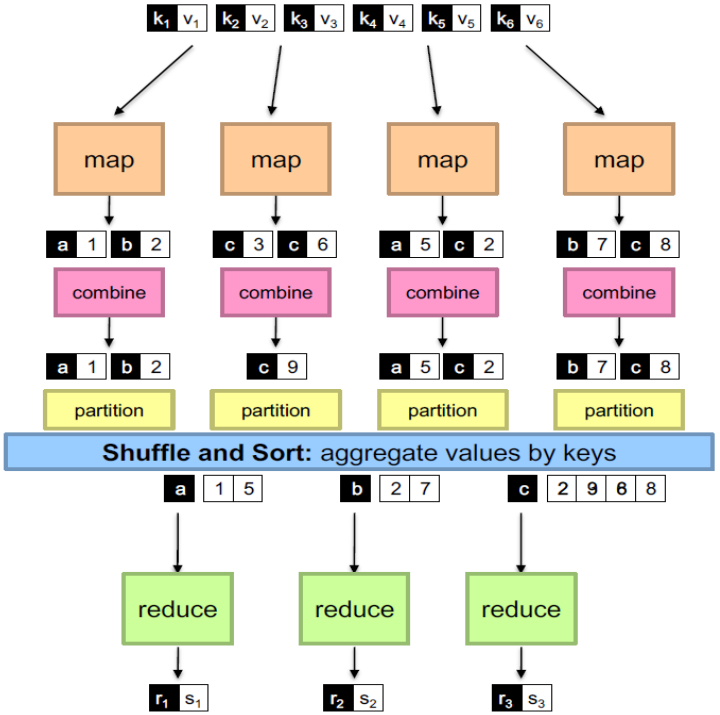
https://data-flair.training/blogs/speculative-execution-in-hadoop-mapreduce/

Handles scheduling – Straggler

Straggler - Two Causes

- Machine suffering from broken hardware
  - Solution: speculative execution (an identical copy of the same task is executed on different machines and uses the result of the first task complete.)
- Skew in the distribution of values associated with intermediate keys

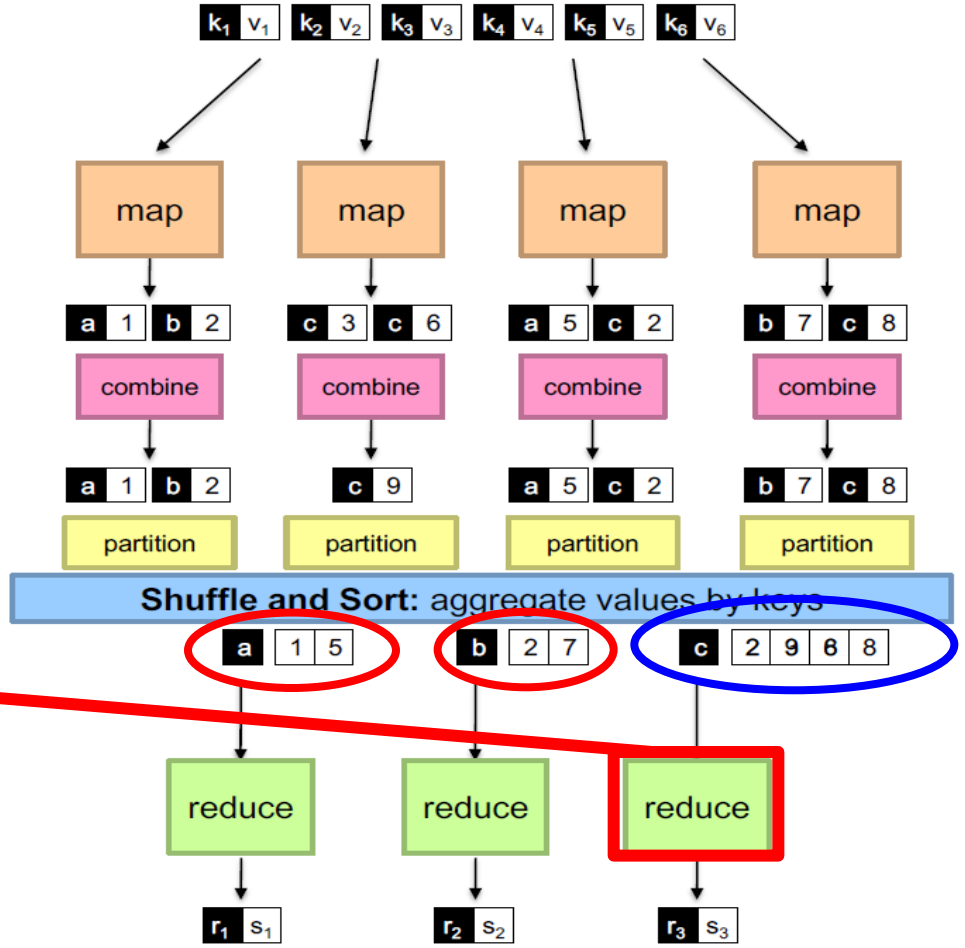
Intermediate data



Handles scheduling – Straggler

- Skew in the distribution of values associated with intermediate keys

3 keys - a, b, c



The reduce task responsible for processing the most frequent elements will run much longer

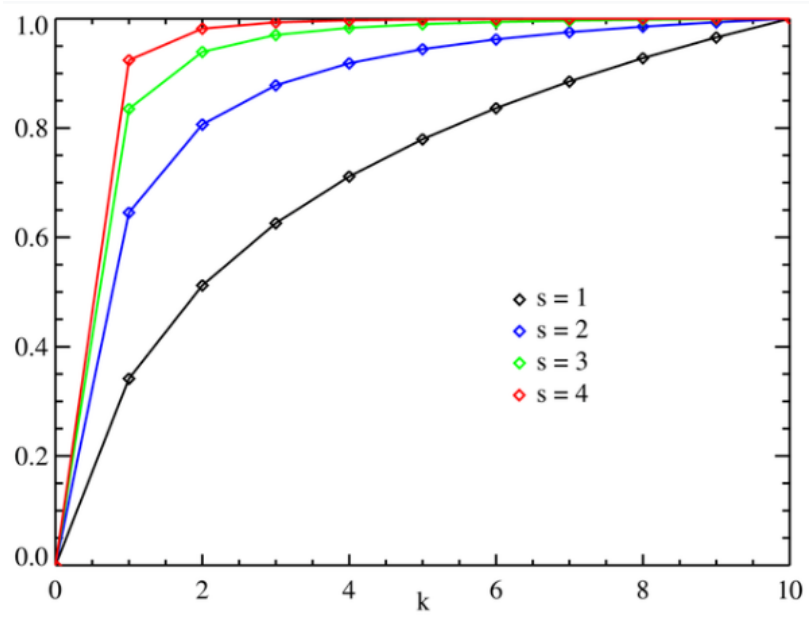
Note, **Zipfian distribution** is common in text processing

Handles scheduling – Straggler

- **Zipfian distribution**

One of a family of related discrete power law probability distributions. Formally, the normalized frequency of elements of rank *k*,

$$f(k; s, N) = \frac{1/k^s}{\sum_{n=1}^{N} (1/n^s)}$$



- *N* be the number of elements;
- *k* be their rank;
- *s* be the value of the exponent characterizing the distribution.

MapReduce Execution Framework transparently handles everything else…

- Handles scheduling
  - Assigns workers to map and reduce tasks
- Handles "data distribution"
  - Moves processes to data
- Handles synchronization
  - Gathers, sorts, and shuffles intermediate data
- Handles errors and faults
  - Detects worker failures and restarts

Optimization – prefer node on the same rack in the datacenter as the node holding the relevant data block, since inter-rack bandwidth is significantly less than intra-rack bandwidth.

Data and Code Co-location

While the key idea behind MapReduce is to move the code not the data, we need data to the code sometimes.

- To achieve data locality, the scheduler starts on the node that holds a particular block of data and move code to the data.
- If this is not possible, new tasks will be started elsewhere (another node), and the data will be streamed over the network.
  - For example, the node already running too many tasks,

MapReduce Execution Framework transparently handles everything else…

- Handles scheduling
  - Assigns workers to map and reduce tasks
- Handles "data distribution"
  - Moves processes to data
- Handles synchronization
  - Gathers, sorts, and shuffles intermediate data
- Handles errors and faults
  - Detects worker failures and restarts

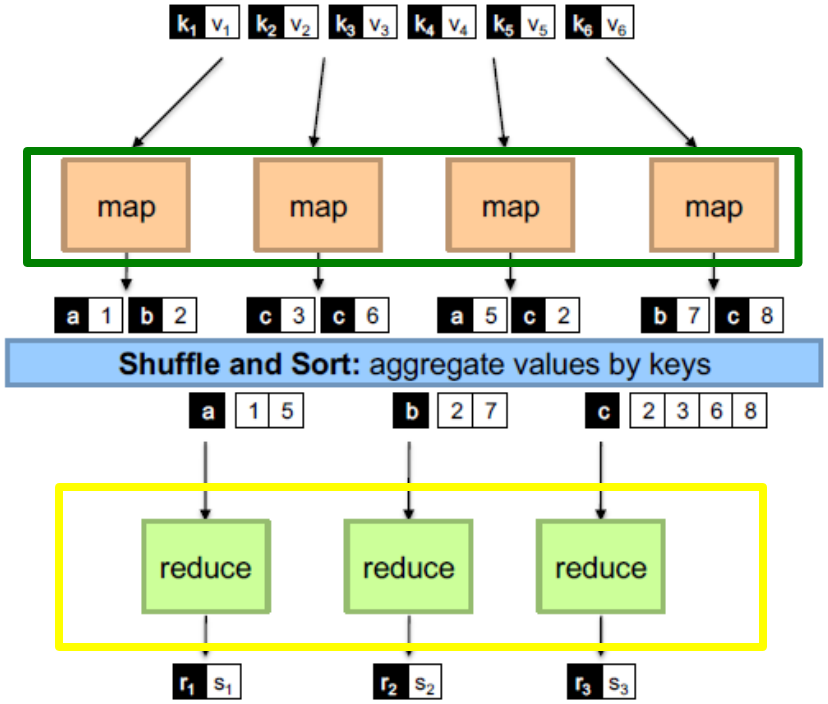
Handles Synchronization

Reduce computation cannot start until all the mappers have finished emitting key-value pairs

- **Synchronization**
  - Enforce the proper execution order when multiple concurrently running processes tries to update the common data.

Synchronization in MapReduce

- Intermediate key-value pairs must be grouped by key. This involves
  - all nodes execute map tasks
  - Copying intermediate data over the network – shuffle and sort
  - all nodes execute reduce tasks



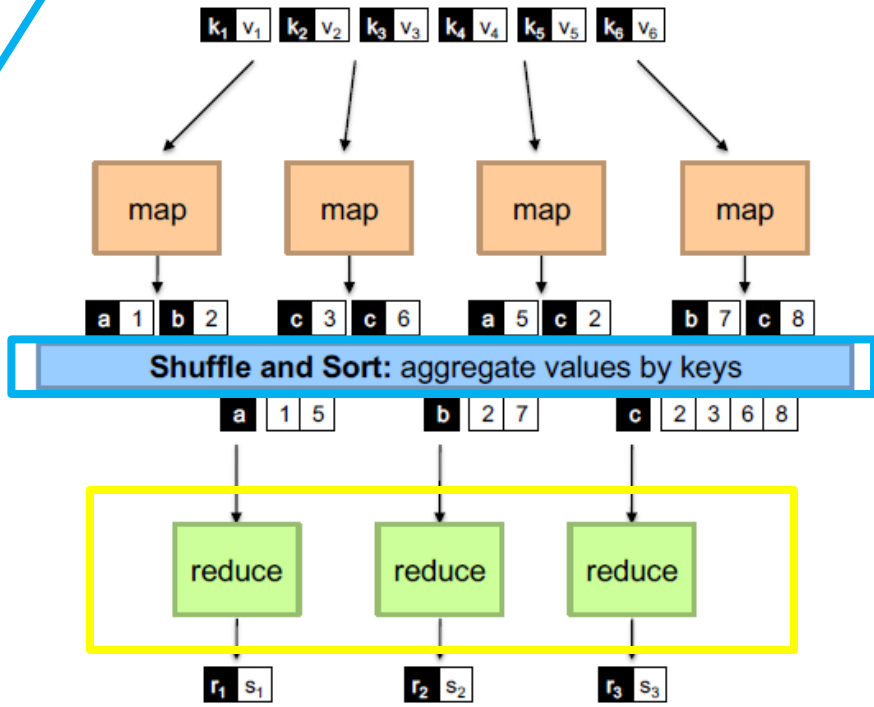
Handles Synchronization

Reduce computation cannot start until all the mappers have finished emitting key-value pairs and all intermediate key-value pairs have been shuffled and sorted

- **Synchronization**
  - Enforce the proper execution order when multiple concurrently running processes tries to update the common data.

Synchronization in MapReduce

- Intermediate key-value pairs must be grouped by key. This involves
  - all nodes execute map tasks
  - Copying intermediate data over the network – shuffle and sort
  - all nodes execute reduce tasks



MapReduce Execution Framework transparently handles everything else…

- Handles scheduling
  - Assigns workers to map and reduce tasks
- Handles "data distribution"
  - Moves processes to data
- Handles synchronization
  - Gathers, sorts, and shuffles intermediate data
- Handles errors and faults
  - Detects worker failures and restarts

Handles Errors and Faults

MapReduce are explicitly designed around low-end commodity servicers

- Hardware
  - disk failures
  - memory failures
  - planned outages (e.g. system maintenance and hardware upgrades)
  - Unexpected outage (e.g. power failure, connectivity loss, etc.)
- Software
  - No software is bug free
  - Solving large problems may fall in extreme cases where bugs trigger unusual errors
  - Large dataset may contain various types of corrupted data or records

The MapReduce execution framework must thrive in this hostile environment

End of Part 2

Part 3
MapReduce Basics

RMIT Classification: Trusted

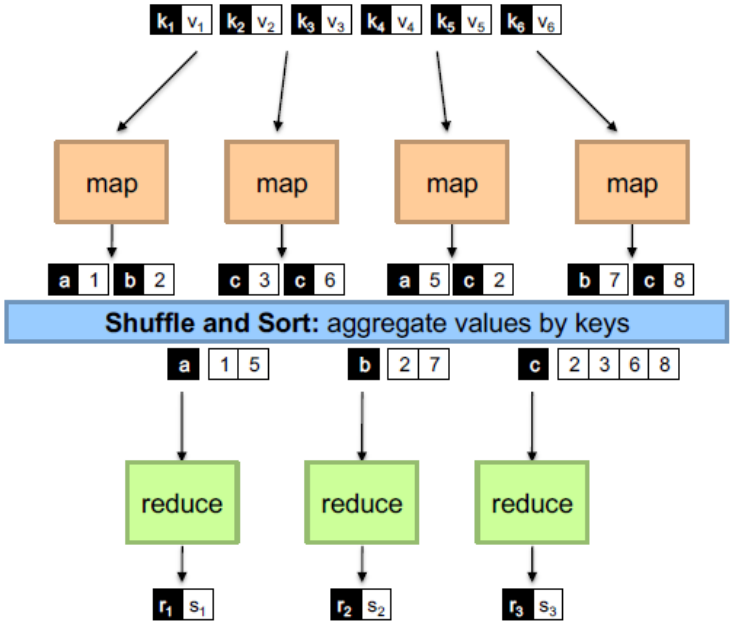
MapReduce

Programmers implement two functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

reduce $(k_2, [v_2]) \rightarrow [<k_3, v_3>]$

All values with the same key are reduced together



RMIT Classification: Trusted

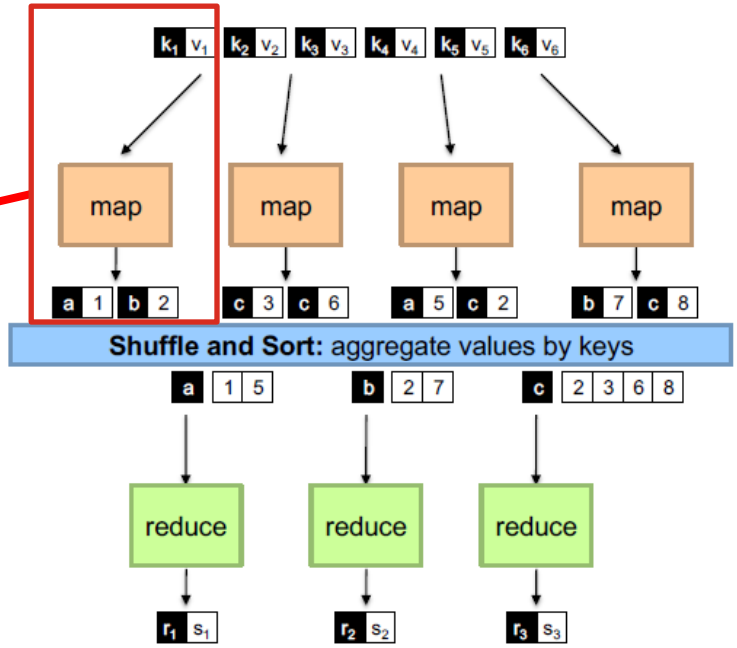
MapReduce

Programmers implement two functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

reduce $(k_2, [v_2]) \rightarrow [<k_3, v_3>]$

All values with the same key are reduced together



(k_1, v_1), [< k_2, v_2>], map task store and process by a container

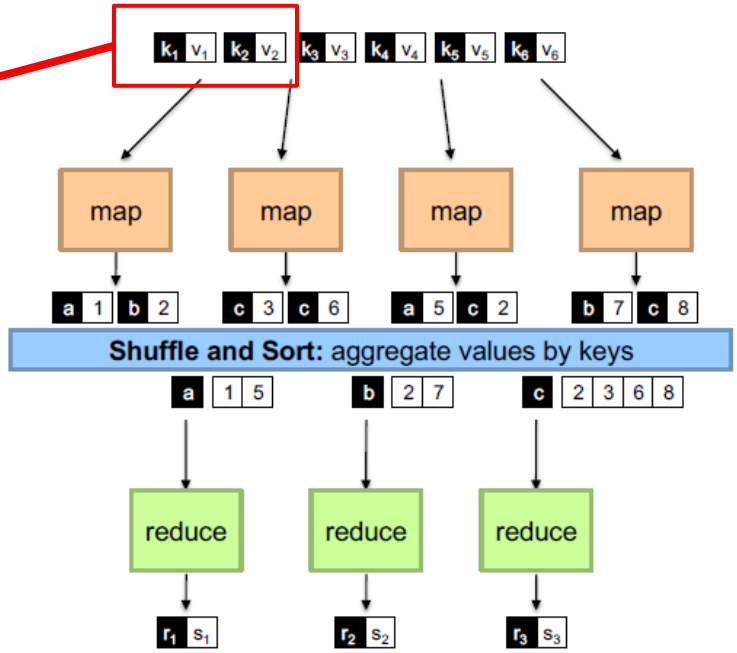
MapReduce

Programmers implement two functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

reduce $(k_2, [v_2]) \rightarrow [<k_3, v_3>]$

All values with the same key are reduced together



(k_1, v_1) is each individual input record of map

key value

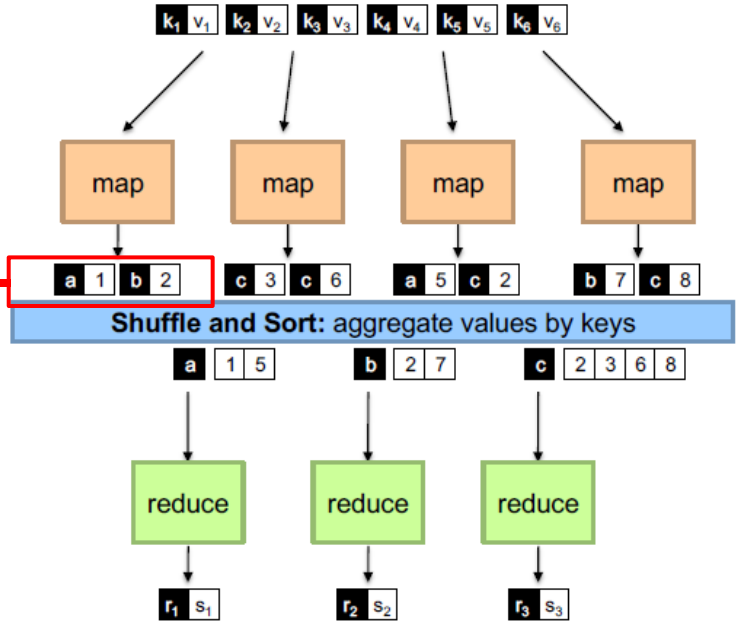
MapReduce

Programmers implement two functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

reduce $(k_2, [v_2]) \rightarrow [<k_3, v_3>]$

All values with the same key are reduced together



[< k_2, v_2 >] is a set of records output by each individual map function

key value

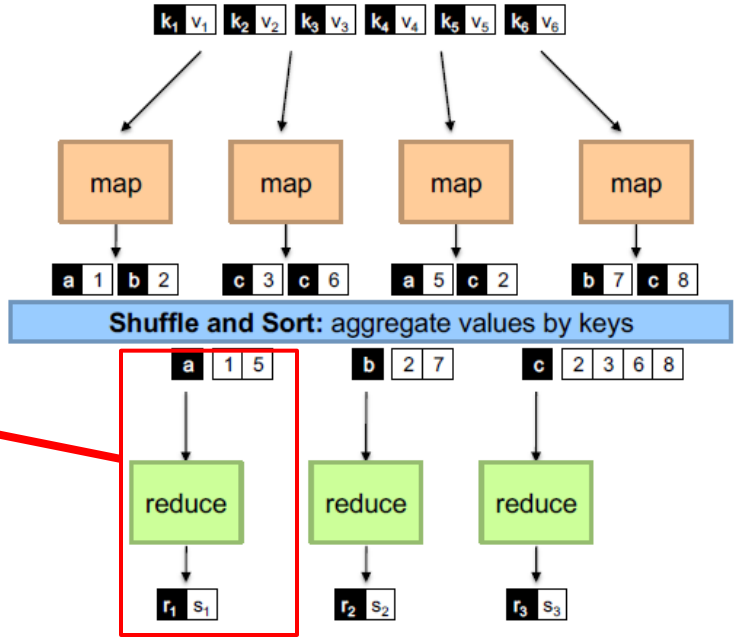
MapReduce

Programmers implement two functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

reduce $(k_2, [v_2]) \rightarrow [<k_3, v_3>]$

All values with the same key are reduced together



$(k_2, [v_2])$, $[<k_3, v_3>]$, reducer task store and process by a container

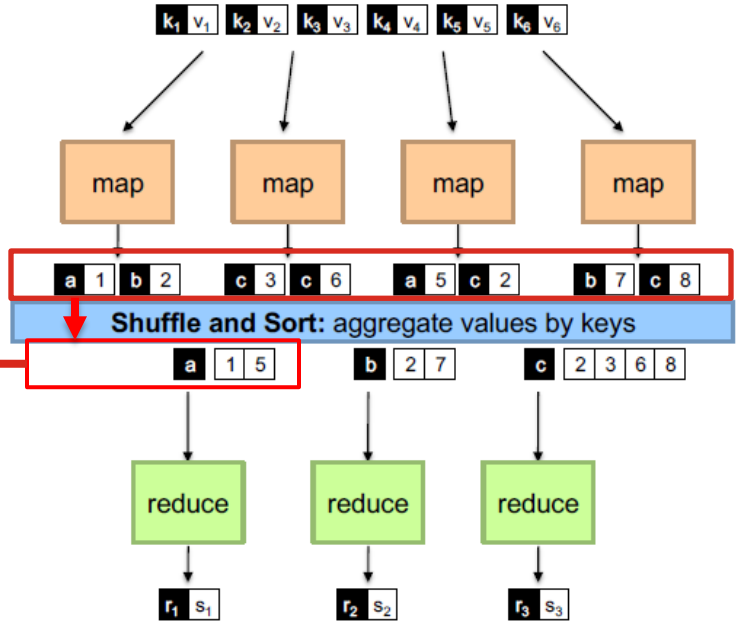
MapReduce

Programmers implement two functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

reduce $(k_2, [v_2]) \rightarrow [<k_3, v_3>]$

All values with the same key are reduced together



$(k_2, [v_2])$ is a record by shuffling & sorting outputs of all map functions

key

a list of values associated with key k_2

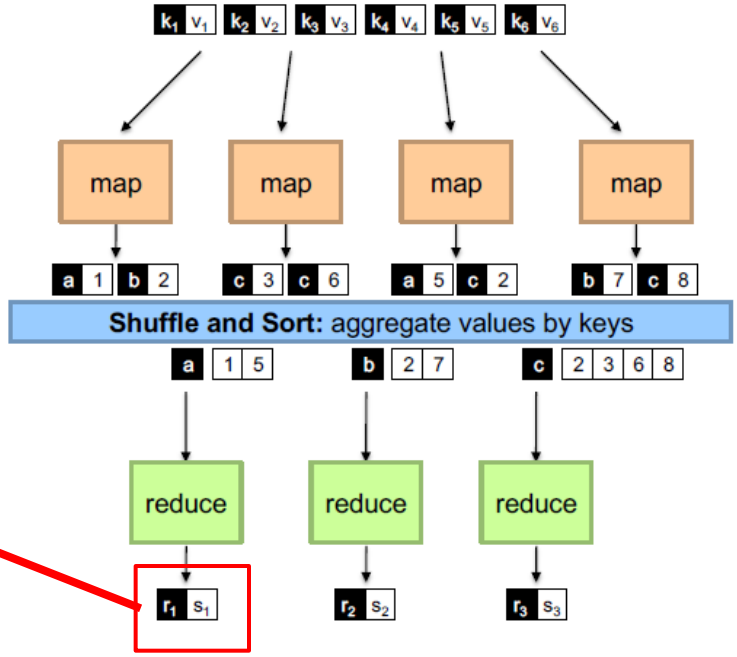
MapReduce

Programmers implement two functions:

map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

reduce $(k_2, [v_2]) \rightarrow [<k_3, v_3>]$

All values with the same key are reduced together



$[<k_3, v_3>]$, is a set of records output by each individual reduce task

key

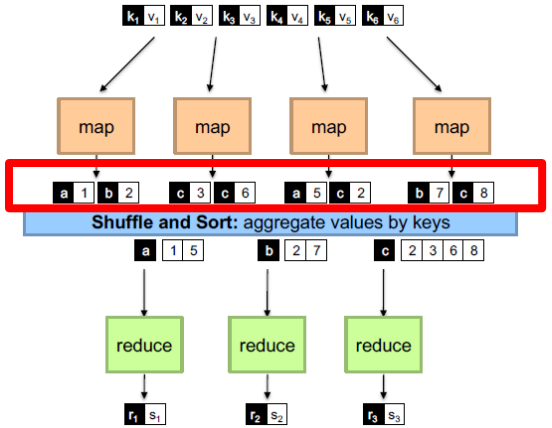
Reduced value associated with key k_3

Example: "map" and "reduce" only

WordCount

Reads text files and counts the frequency of the words.

- Each mapper takes a line of the input file as input and breaks it into words.
- It then emits a key/value pair for each word like (word, 1)



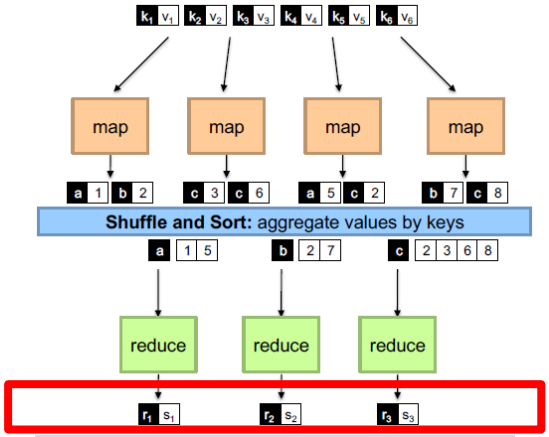
For example, given a text file:

Hello wordcount MapReduce Hadoop program. This is my first MapReduce program

Hello	*1*
wordcount	*1*
MapReduce	*1*
Hadoop	*1*
program	*1*
This	*1*
is	*1*
my	*1*
first	*1*
MapReduce	*1*
program	*1*

Example: "map" and "reduce" only

first	*1*
Hadoop	*1*
Hello	*1*
is	*1*
MapReduce	*2*
my	*1*
program	*2*
This	*1*
wordcount	*1*



Hello	*1*
wordcount	*1*
MapReduce	*1*
Hadoop	*1*
program	*1*
This	*1*
is	*1*
my	*1*
first	*1*
MapReduce	*1*
program	*1*

- Each reducer sums the counts for each word and emits a single key/value with the word and sum.

For example, given a text file:

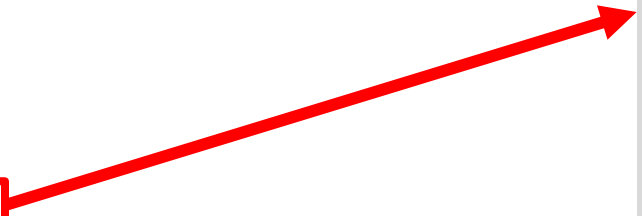
Hello wordcount MapReduce Hadoop program. This is my first MapReduce programz

Example: "map" and "reduce" only

Word Count mapper: python

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
  line = line.strip()
  words = line.split(" ")
  for word in words:
    print('%s\t%s' % (word,1))
```



Hello	*1*
wordcount	*1*
MapReduce	*1*
Hadoop	*1*
program	*1*
This	*1*
is	*1*
my	*1*
first	*1*
MapReduce	*1*
program	*1*

Example: "map" and "reduce" only

Word Count reducer: python

```
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word,count = line.split('\t',1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word
if current_word == word:
    print('%s\t%s' % (current_word,current_count))
```

Hello	*1*
wordcount	*1*
MapReduce	*1*
Hadoop	*1*
program	*1*
This	*1*
is	*1*
my	*1*
first	*1*
MapReduce	*1*
program	*1*

Example: "map" and "reduce" only

Word Count reducer: python

```
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word,count = line.split('\t',1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word
if current_word == word:
    print('%s\t%s' % (current_word,current_count))
```

Hello	*1*
wordcount	*1*
MapReduce	*1*
Hadoop	*1*
program	*1*
This	*1*
is	*1*
my	*1*
first	*1*
MapReduce	*1*
program	*1*

first	*1*
Hadoop	*1*
Hello	*1*
is	*1*
MapReduce	*2*
my	*1*
program	*2*
This	*1*
wordcount	*1*

MapReduce

Programmers implement two functions:

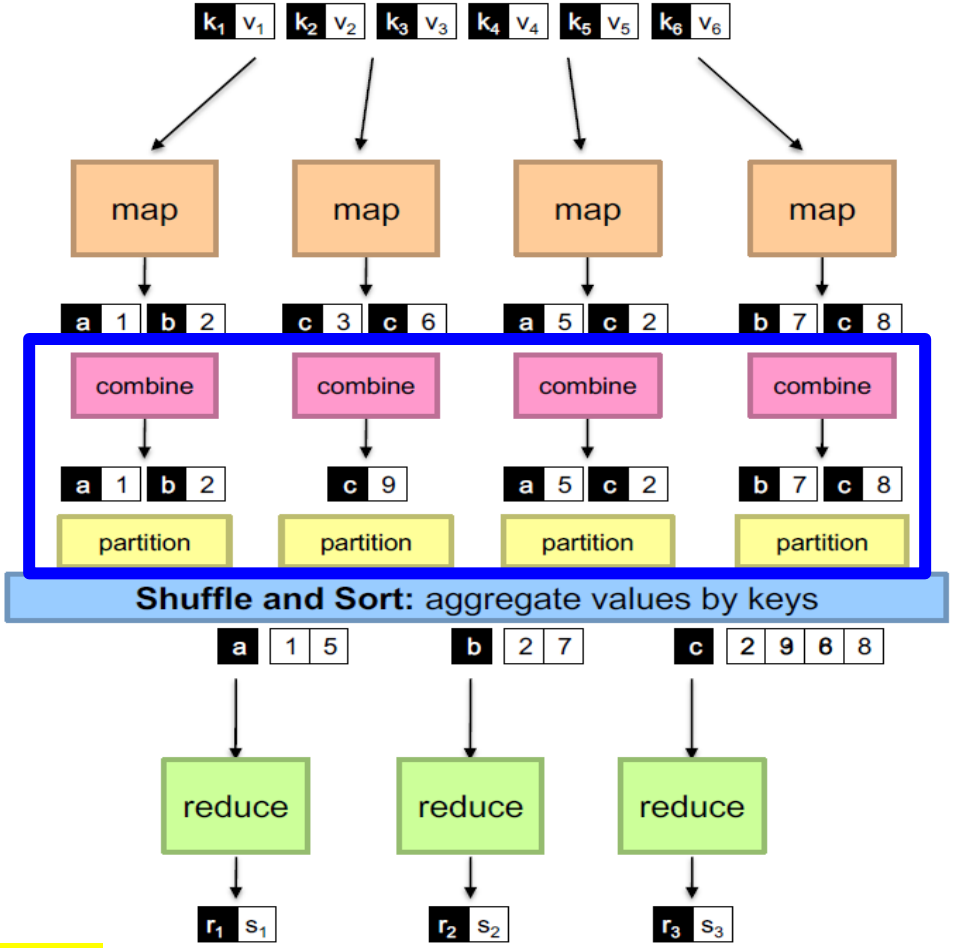
map $(k_1, v_1) \rightarrow [<k_2, v_2>]$

reduce $(k_2, [v_2]) \rightarrow [<k_3, v_3>]$

All values with the same key are reduced together

Optional functions implemented by programmers:

- **combine** $(k_2, [v_2]) \rightarrow [< k_2, v_2 >]$
- **partition** $(k_2, \#\text{ of partitions}) \rightarrow$ partition for k_2



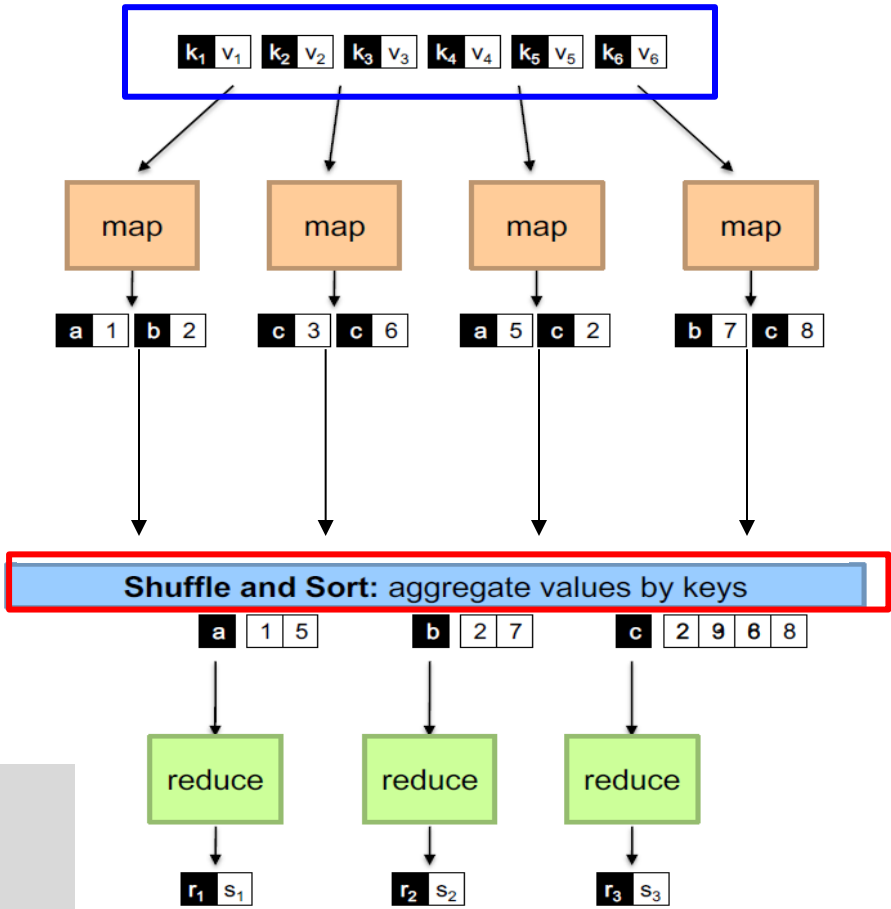
Why we need combine?

MapReduce- Combiners

If not implement combiner

All emitted key-value pairs from mappers need to be copied across the network

The amount of intermediate data will be larger than the input data. This is inefficient.



Hello wordcount MapReduce Hadoop program. This is my first MapReduce program

Hello	*1*
wordcount	*1*
MapReduce	*1*
Hadoop	*1*
program	*1*
This	*1*
is	*1*
my	*1*
first	*1*
MapReduce	*1*
program	*1*

MapReduce- Combiners

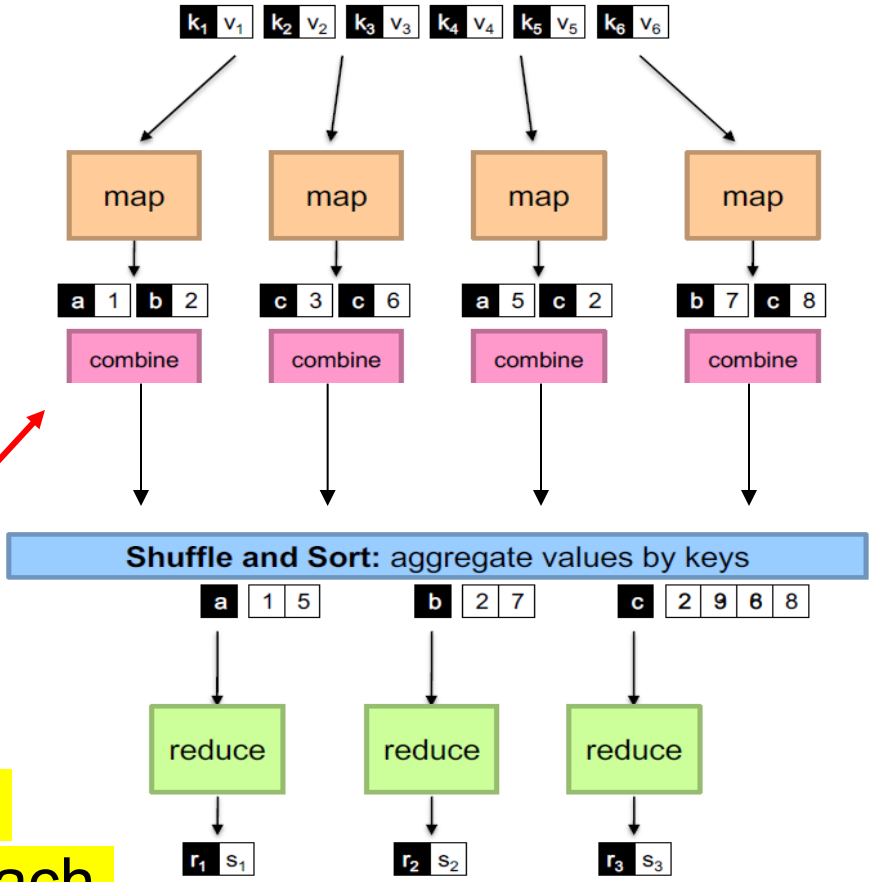
If not implement combiner

All emitted key-value pairs from mappers need to be copied across the network

The amount of intermediate data will be larger than the input data. This is inefficient.

Solution

Before the shuffle and sort, perform local aggregation on the output of each mapper - combiner



MapReduce- Combiners

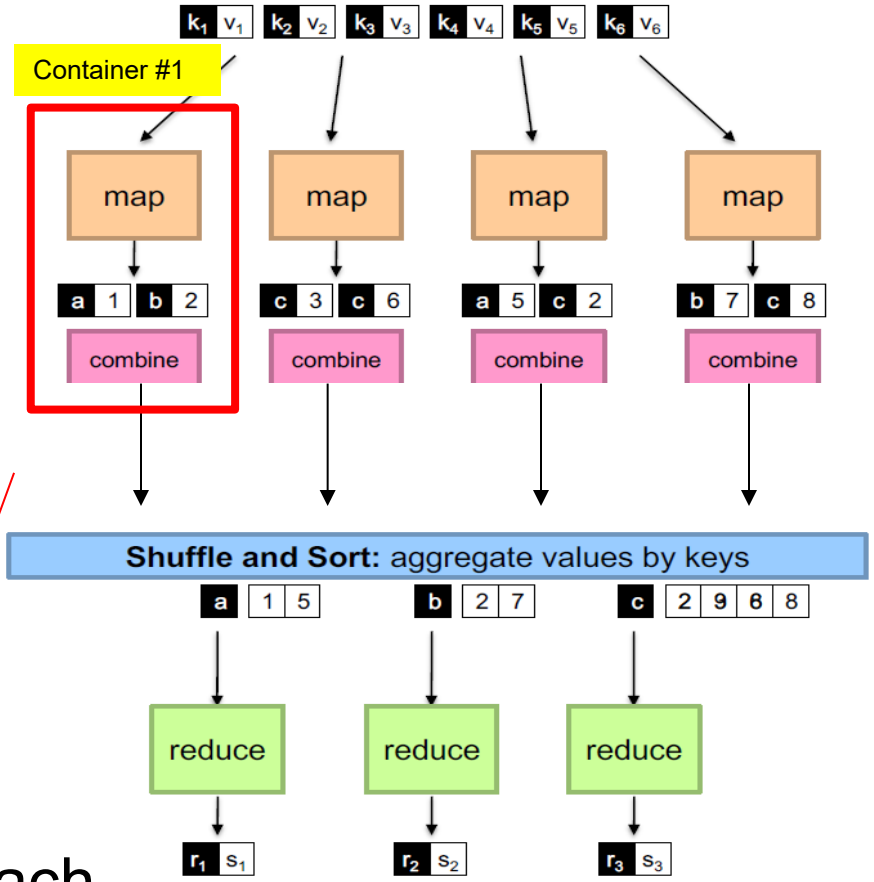
If not implement combiner

All emitted key-value pairs from mappers need to be copied across the network

So the amount of intermediate data will be larger than the input data. This is inefficient.

Solution

Before the shuffle and sort, perform local aggregation on the output of each mapper - combiner

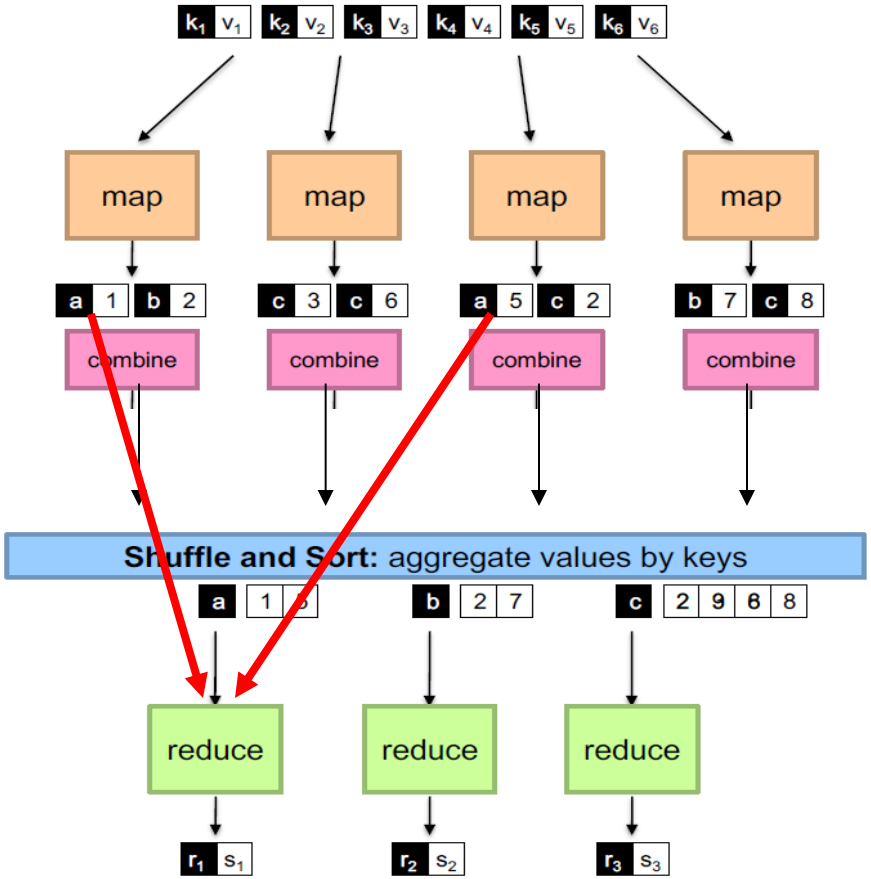


Combiner - compute a local count for a key over all data processed by the associated map task.

Partition, Shuffle and Sorting

- **Partition & Shuffle**

After the map tasks have completed, it begins exchanging the intermediate outputs from the map tasks to where they are required by the reducers.



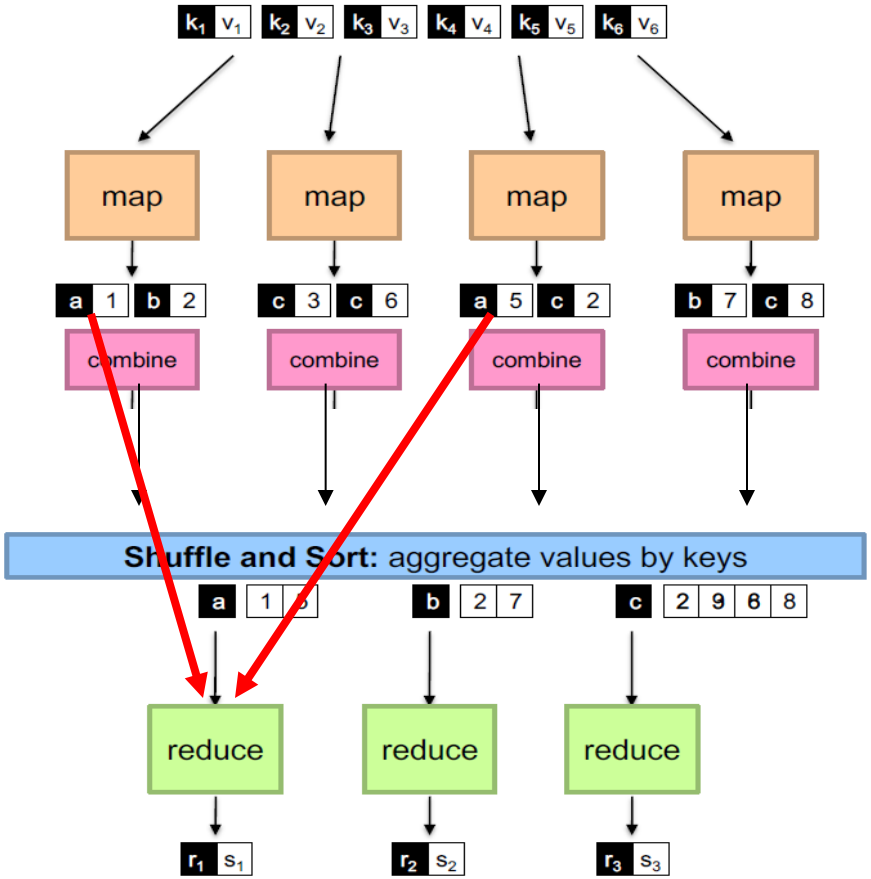
This process of moving map outputs to the reducers is known as *shuffling*.

All values with the same key are always reduced together regardless of which mapper is its origin.

Partition, Shuffle and Sorting

- **Partition & Shuffle**

After the map tasks have completed, it begins exchanging the intermediate outputs from the map tasks to where they are required by the reducers.



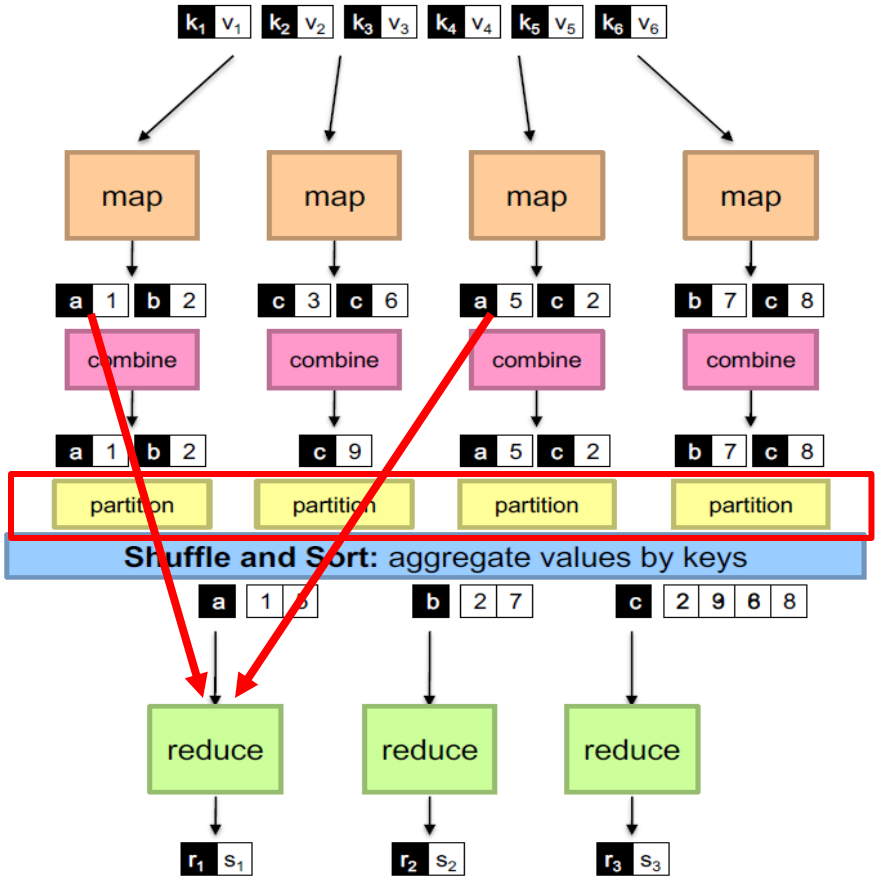
This process of moving map outputs to the reducers is known as *shuffling*.

For each (key, value) pair, MapReduce platform decides which partition it should go.

Partition, Shuffle and Sorting

- **Partition & Shuffle**

After the map tasks have completed, it begins exchanging the intermediate outputs from the map tasks to where they are required by the reducers.



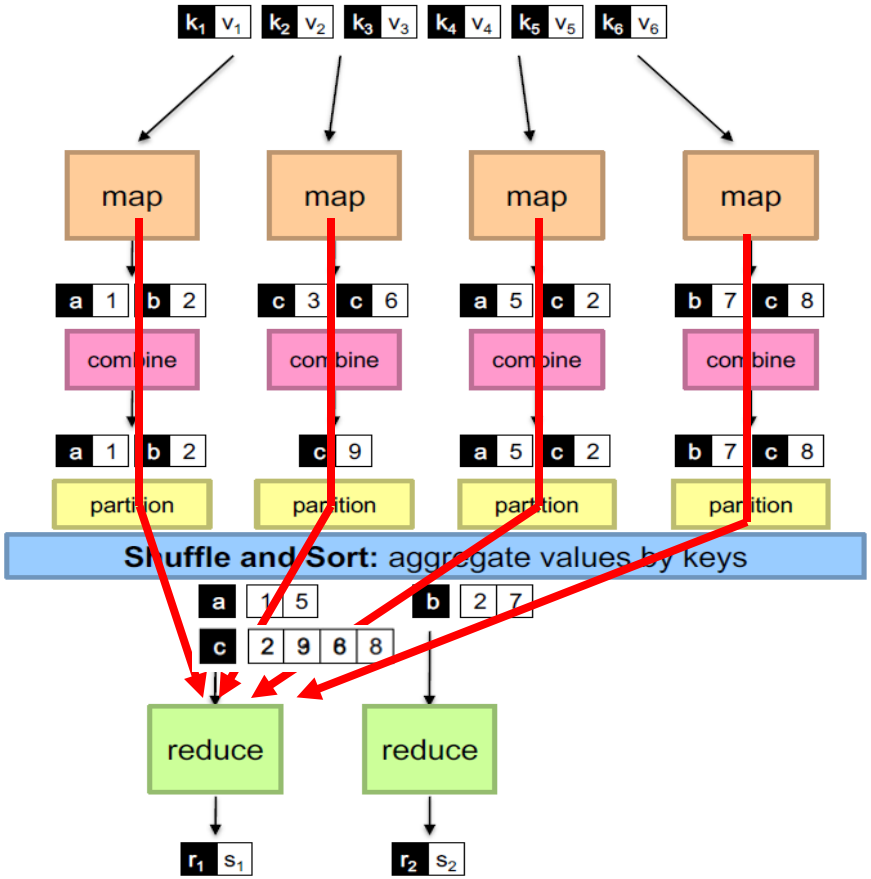
This process of moving map outputs to the reducers is known as *shuffling*.

For each (key, value) pair, if a programmer wants to determine which partition it will go, implement partitioner.

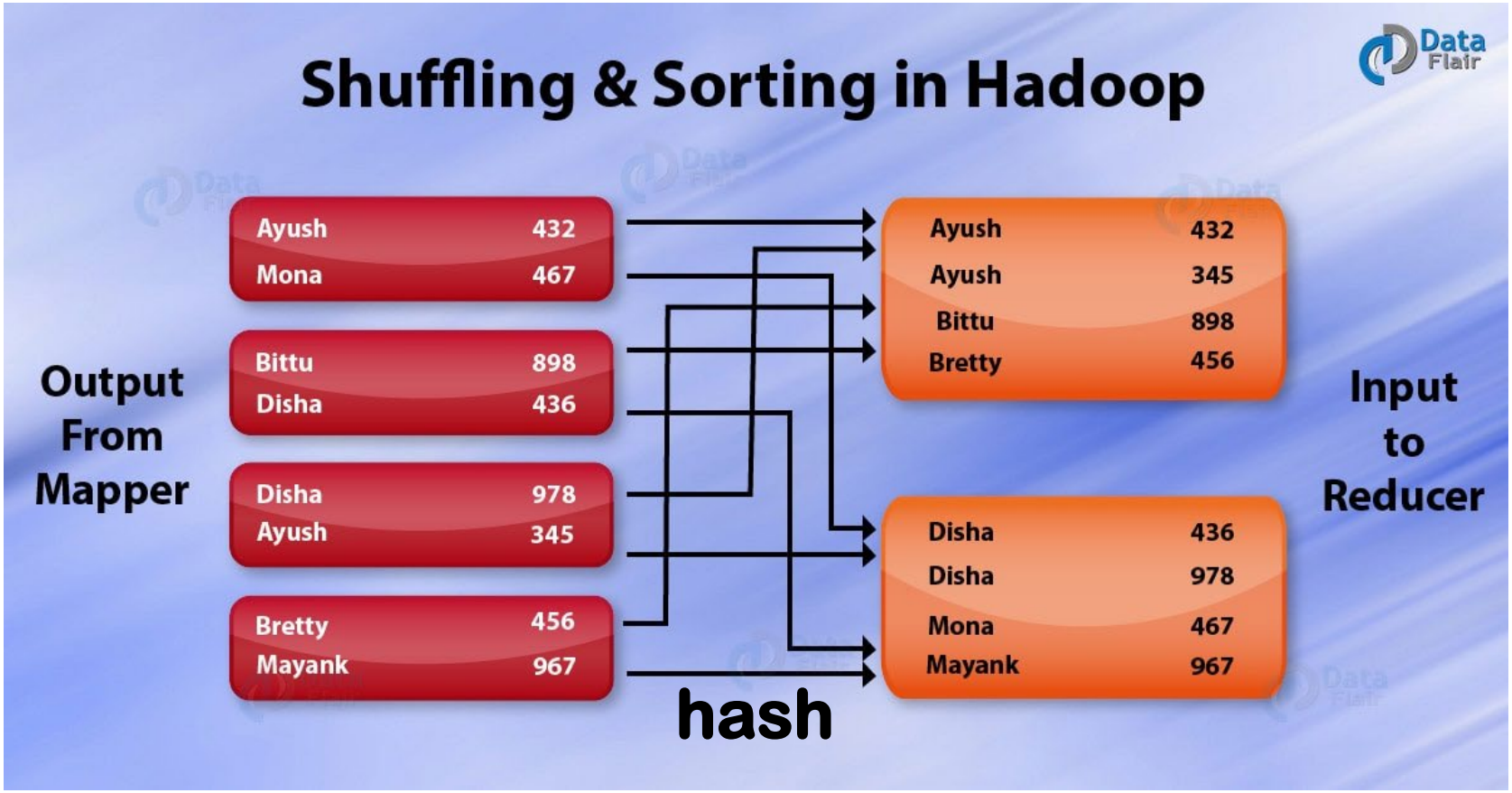
Partition, Shuffle and Sorting

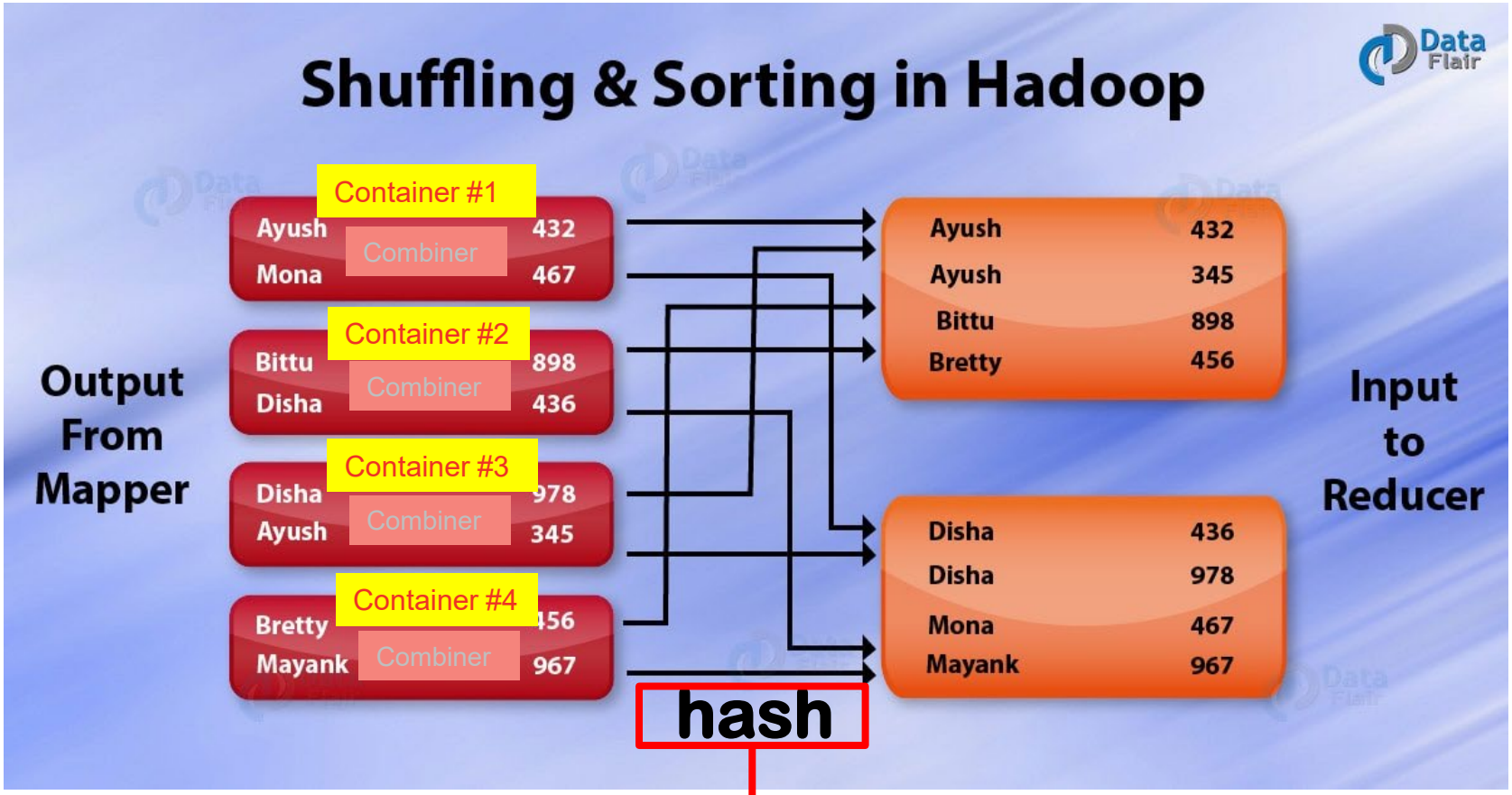
- **Sorting**

Before starting of reducer, all intermediate key-value pairs in MapReduce that are generated by mapper get sorted by key and not by value. Values passed to each reducer are not sorted; they can be in any order.



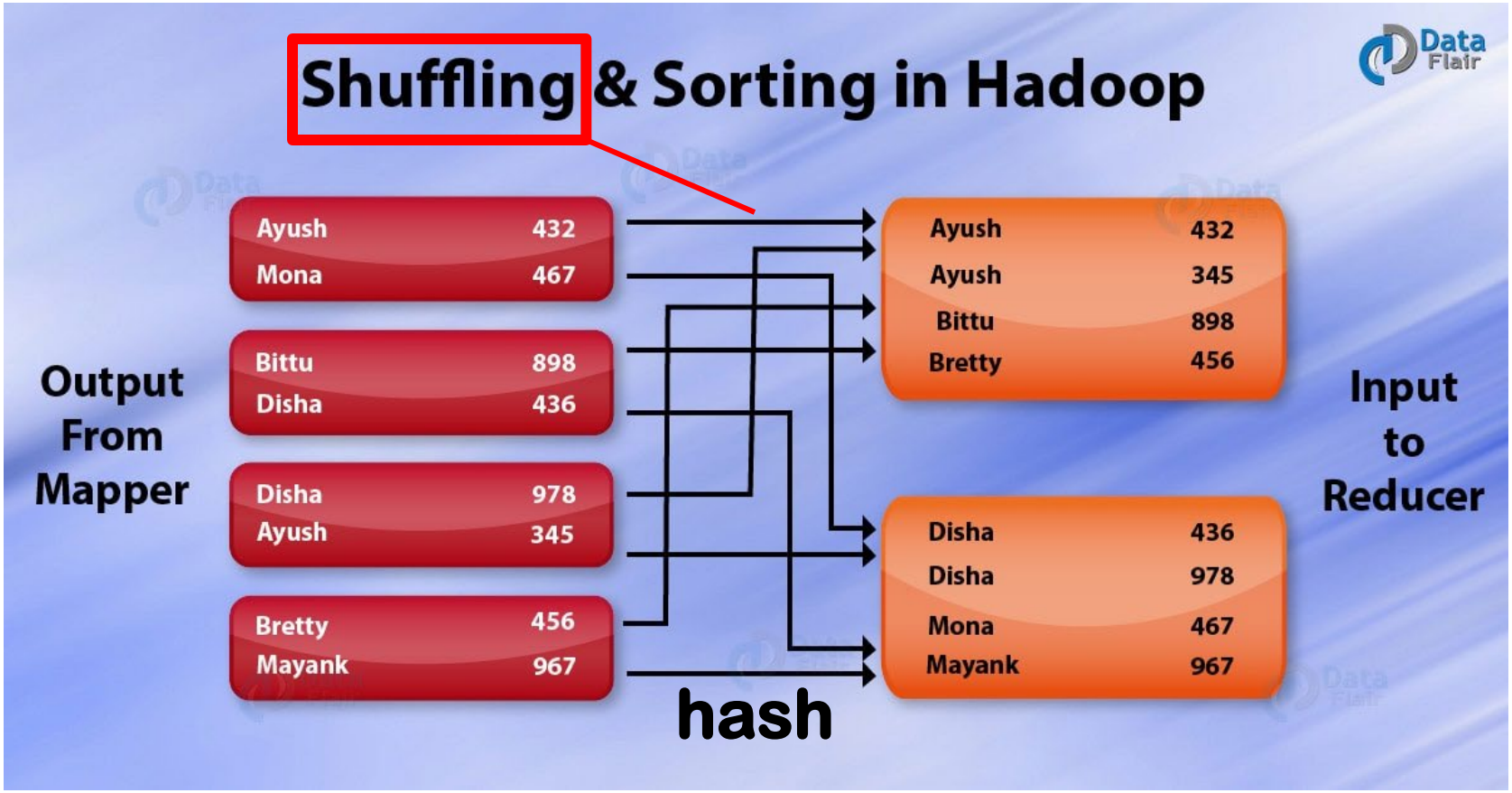
Assume there are two reducers where key a and key c are shuffled to the same reducer

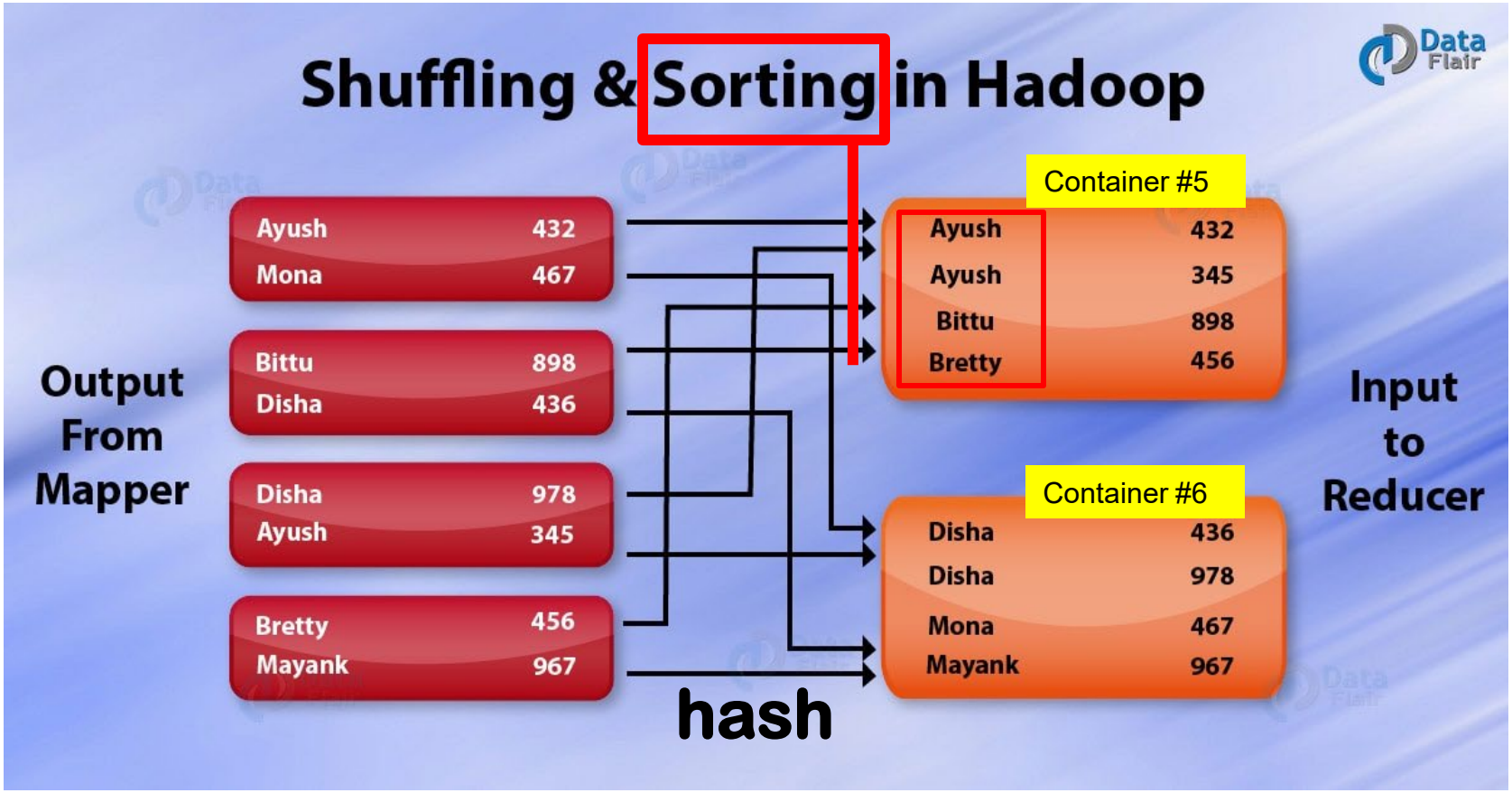

Shuffling & Sorting in Hadoop
Data Flair
Output From Mapper
Ayush 432
Mona 467
Bittu 898
Disha 436
Disha 978
Ayush 345
Bretty 456
Mayank 967
hash
Ayush 432
Ayush 345
Bittu 898
Bretty 456
Disha 436
Disha 978
Mona 467
Mayank 967
Input to Reducer



Partition

The default partitioner computes a hash value for the key and assigns the partition based on this result.


Shuffling & Sorting in Hadoop
Data Flair
Output From Mapper
Ayush 432
Mona 467
Bittu 898
Disha 436
Disha 978
Ayush 345
Bretty 456
Mayank 967
hash
Input to Reducer
Ayush 432
Ayush 345
Bittu 898
Bretty 456
Disha 436
Disha 978
Mona 467
Mayank 967

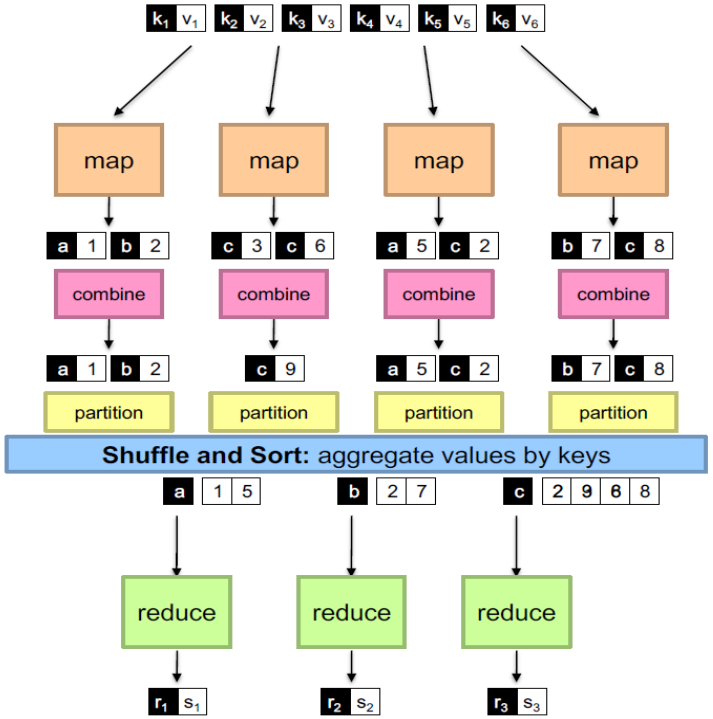

Shuffling & Sorting in Hadoop
Data Flair
Output From Mapper
Ayush 432
Mona 467
Bittu 898
Disha 436
Disha 978
Ayush 345
Bretty 456
Mayank 967
hash
Container #5
Ayush 432
Ayush 345
Bittu 898
Bretty 456
Container #6
Disha 436
Disha 978
Mona 467
Mayank 967
Input to Reducer

A summary - as a programmer,

- You have limited control over data and execution flow!
  - All algorithms must be expressed in

"***map, reduce, combine, partition***"

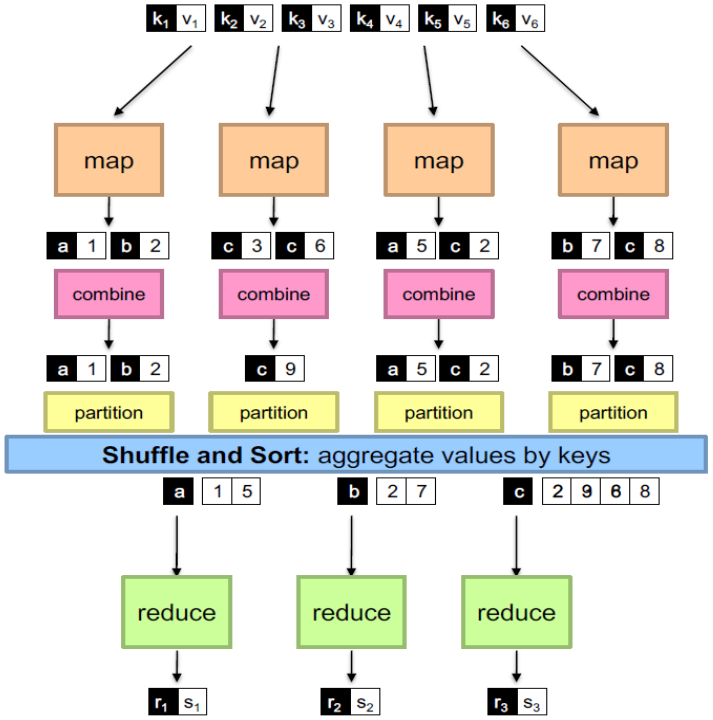
- You don't know:
  - Where mappers and reducers run
  - When a mapper or reducer begins or finishes
  - Which input a particular mapper is processing
  - Which intermediate key a particular reducer is processing



A summary - as a programmer,

- You have limited control over data and execution flow!
  - All algorithms must be expressed in

"***map, reduce, combine, partition***"



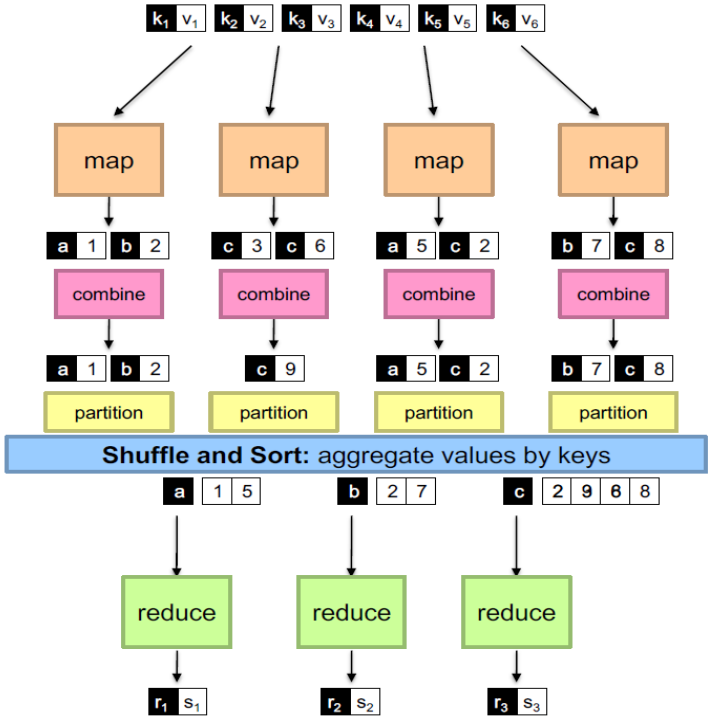
- You don't know:
  - Where mappers and reducers run
  - When a mapper or reducer begins or finishes
  - Which input a particular mapper is processing
  - Which intermediate key a particular reducer is processing

Resource Manager, Application Master, Node Manager, Containers

A summary - as a programmer,

- You have limited control over data and execution flow!
  - All algorithms must be expressed in

"***map, reduce, combine, partition***"



- You don't know:
  - Where mappers and reducers run
  - When a mapper or reducer begins or finishes
  - Which input a particular mapper is processing
  - Which intermediate key a particular reducer is processing

Resource Manager, Application Master, Node Manager, Containers

one container

one MapReduce task (map task, reduce task)

End of Part 3

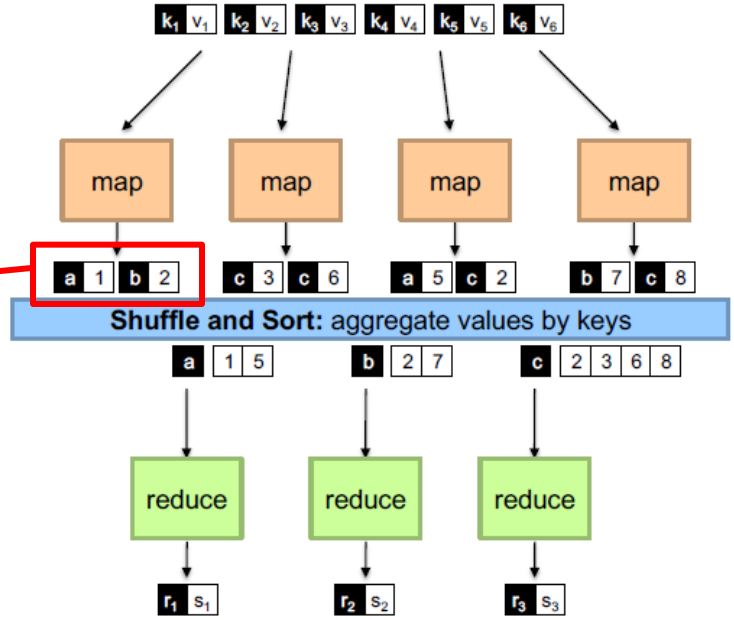
Part 4

MapReduce Local Aggregation

Local Aggregation

- Transfer data over network

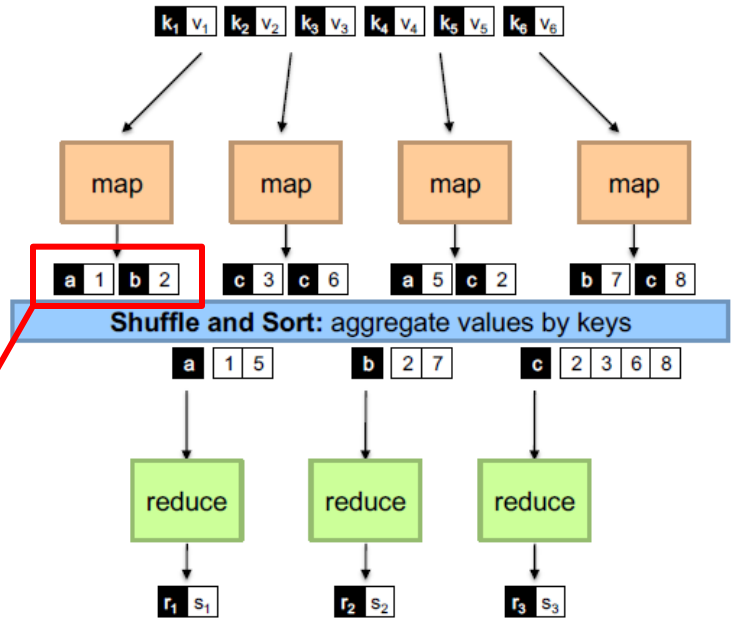
Exchange of intermediate results from the processes (i.e., mapper) that produced them to the processes (i.e., reducer) that will ultimately consume them.



Local Aggregation

- Transfer data over network

Exchange of intermediate results from the processes (i.e., mapper) that produced them to the processes (i.e., reducer) that will ultimately consume them.



Data written to local disk

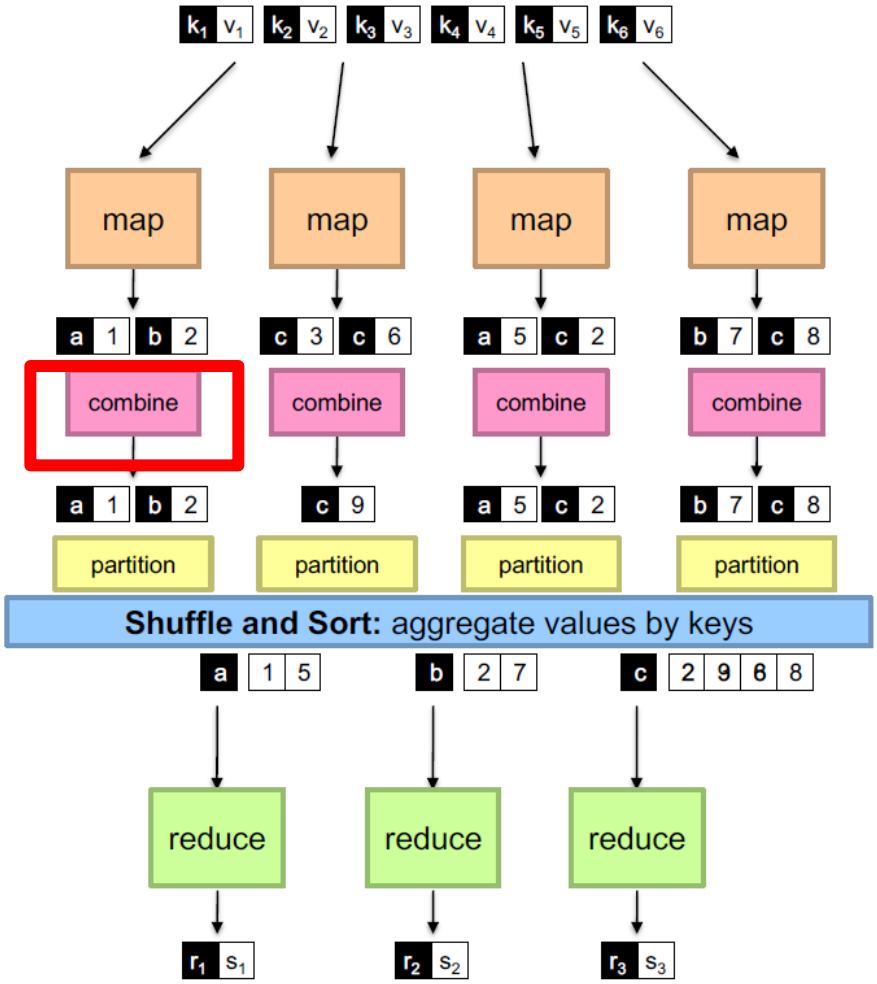
Intermediate results are sorted and written to local disk before being sent over the network.

Local aggregation of intermediate results is on the keys

- it uses in-mapper combining or combiner
- it preserves state across multiple inputs (i.e., across lines)

Combiner and In-mapper Combining

- **Combiner (introduced already)**
  - Combiners aggregate term counts of intermediate key-value pairs by each map task.



Combiner and In-mapper Combining

Hello wordcount MapReduce Hadoop program. This is my first MapReduce program

mapper.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
  line = line.strip()
  words = line.split(" ")
  for word in words:
    print('%s\t%s' % (word,1))
```

Combiner An Example

Combiner and In-mapper Combining

Hello wordcount MapReduce Hadoop program. This is my first MapReduce program

mapper.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
  line = line.strip()
  words = line.split(" ")
  for word in words:
    print('%s\t%s' % (word,1))
```

write to local disk

Combiner An Example

Hello	*1*
wordcount	*1*
MapReduce	*1*
Hadoop	*1*
program	*1*
This	*1*
is	*1*
my	*1*
first	*1*
MapReduce	*1*
program	*1*

read local disk

Combiner

You can specify the reducer as combiner

write to local disk

first	*1*
Hadoop	*1*
Hello	*1*
is	*1*
MapReduce	*2*
my	*1*
program	*2*
This	*1*
wordcount	*1*

Hadoop makes no guarantees on how many times the combiner is called, it could be 0, 1, or multiple times

Combiner and In-mapper Combining

- **In-mapper Combining**

With this technique, the combiner functionality has been directly applied insider mapper; no need to run a separate combiner.

Combining

- **In-mapper Combining**

mapper.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
  line = line.strip()
  words = line.split(" ")
  for word in words:
    print('%s\t%s' % (word,1))
```

Mapper-in-mapper-combining.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    mylist = [];
    line = line.strip()
    words = line.split(" ")
    for word in words:
        flag = False
        i = 0
        for x in mylist:
            item = x.split(" ")
            w = item[0]
            n = item[1]
            if word == w:
                nn = int(n) + 1
                mylist[i] = w + " " + str(nn)
                flag = True
                break
            i = i + 1

        if not flag:
            mylist.append(word+ " " + "1")

    for x in mylist:
        item = x.split(" ")
        print('%s\t%s' % (item[0],item[1]))
```

A list data structure is introduced inside the mapper to tally up term counts within a single document

Combiner and In-mapper Combining

Hello wordcount MapReduce Hadoop program. This is my first MapReduce program

written to local disk

Hello	*1*
wordcount	*1*
MapReduce	*1*
Hadoop	*1*
program	*1*
This	*1*
is	*1*
my	*1*
first	*1*
MapReduce	*1*
program	*1*

Mapper-in-mapper-combining.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    mylist = [];
    line = line.strip()
    words = line.split(" ")
    for word in words:
        flag = False
        i = 0
        for x in mylist:
            item = x.split(" ")
            w = item[0]
            n = item[1]
            if word == w:
                nn = int(n) + 1
                mylist[i] = w + " " + str(nn)
                flag = True
                break
            i = i + 1

        if not flag:
            mylist.append(word+ " " + "1")

    for x in mylist:
        item = x.split(" ")
        print('%s\t%s' % (item[0],item[1]))
```

Can we do even better?

Combiner and In-mapper Combining

- **In-mapper Combining (preserve state across lines)**
  - The mapper Initializes a list prior to processing any input key-value pairs
    - Initialize an associative array for holding term counts

Combiner and In-mapper Combining

- **In-mapper Combining (preserve state across lines)**
  - The mapper Initializes a list prior to processing any input key-value pairs
    - Initialize an associative array for holding term counts

In-mapper Combining (preserve state across lines)

```
#!/usr/bin/env python3
import sys
mylist = [];
for line in sys.stdin:
    line = line.strip()
    words = line.split(" ")
    for word in words:
        flag = False
        i = 0
        for x in mylist:
            item = x.split(" ")
            w = item[0]
            n = item[1]
            if word == w:
                nn = int(n) + 1
                mylist[i] = w + " " + str(nn)
                flag = True
                break
            i = i + 1

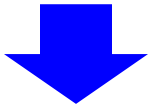
        if not flag:
            mylist.append(word+ " " + "1")

for x in mylist:
    item = x.split(" ")
    print('%s\t%s' % (item[0],item[1]))
```

Combiner and In-mapper Combining

In-mapper Combining (preserve state across lines)

Hello wordcount MapReduce Hadoop program. This is my first MapReduce program



written to local disk

first	*1*
Hadoop	*1*
Hello	*1*
is	*1*
MapReduce	*2*
my	*1*
program	*2*
This	*1*
wordcount	*1*

```
#!/usr/bin/env python3
import sys
mylist = [];
for line in sys.stdin:
    line = line.strip()
    words = line.split(" ")
    for word in words:
        flag = False
        i = 0
        for x in mylist:
            item = x.split(" ")
            w = item[0]
            n = item[1]
            if word == w:
                nn = int(n) + 1
                mylist[i] = w + " " + str(nn)
                flag = True
                break
            i = i + 1

        if not flag:
            mylist.append(word+ " " + "1")

for x in mylist:
    item = x.split(" ")
    print('%s\t%s' % (item[0],item[1]))
```

In-mapper Combining (within each line)

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    mylist = [];
    line = line.strip()
    words = line.split(" ")
    for word in words:
        flag = False
        i = 0
        for x in mylist:
            item = x.split(" ")
            w = item[0]
            n = item[1]
            if word == w:
                nn = int(n) + 1
                mylist[i] = w + " " + str(nn)
                flag = True
                break
            i = i + 1

        if not flag:
            mylist.append(word+ " " + "1")

    for x in mylist:
        item = x.split(" ")
        print('%s\t%s' % (item[0],item[1]))
```

In-mapper Combining (across lines)

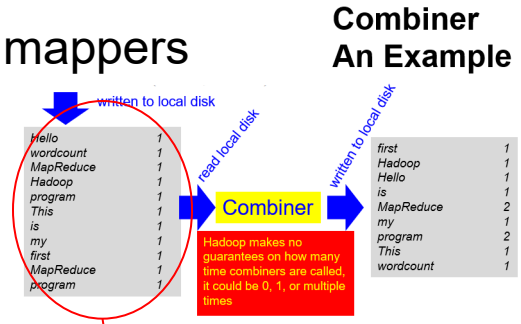
```
#!/usr/bin/env python3
import sys
mylist = [];
for line in sys.stdin:
    line = line.strip()
    words = line.split(" ")
    for word in words:
        flag = False
        i = 0
        for x in mylist:
            item = x.split(" ")
            w = item[0]
            n = item[1]
            if word == w:
                nn = int(n) + 1
                mylist[i] = w + " " + str(nn)
                flag = True
                break
            i = i + 1

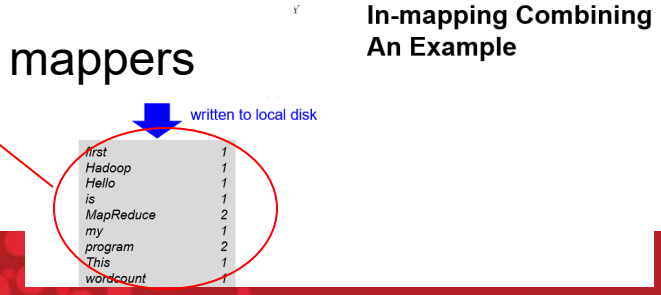
        if not flag:
            mylist.append(word+ " " + "1")

for x in mylist:
    item = x.split(" ")
    print('%s\t%s' % (item[0],item[1]))
```

In-mapper combining
Advantage and Disadvantage

- **Advantage**
  - Provide programmers control over
    - when local aggregation occurs
    - how it is exactly take place
  - More efficient than using actual combiners
    - Combiners reduce intermediate data shuffled across network but don't reduce the number of key-value pairs emitted by the mappers in the first place.
    - With in-mapper combining, the mappers will generate only those key-value pairs need to be shuffled across the network





In-mapper combining Advantage and Disadvantage

- **Disadvantage -** Scalability bottleneck
  - Memory size

It requires sufficient memory to store intermediate results until the mapper has completely processed all key-value pairs in an input split.

- Solution to limited memory, periodically
  - "block" input key-value pairs
  - "flush" in-memory data structure

```
#!/usr/bin/env python3
import sys
mylist = [];
for line in sys.stdin:
    line = line.strip()
    words = line.split(" ")
    for word in words:
        flag = False
        i = 0
        for x in mylist:
            item = x.split(" ")
            w = item[0]
            n = item[1]
            if word == w:
                nn = int(n) + 1
                mylist[i] = w + " " + str(nn)
                flag = True
                break
            i = i + 1

        if not flag:
            mylist.append(word+ " " + "1")

for x in mylist:
    item = x.split(" ")
    print('%s\t%s' % (item[0],item[1]))
```

In-mapper combining
Advantage and Disadvantage

- **Disadvantage -** Scalability bottleneck
  - Memory size

It requires sufficient memory to store intermediate results until the mapper has completely processed all key-value pairs in an input split.

- Solution to limited memory, periodically
  - "block" input key-value pairs
  - "flush" in-memory data structure

```
#!/usr/bin/env python3
import sys
mylist = [];
for line in sys.stdin:
    line = line.strip()
    words = line.split(" ")
```

Idea: instead of emitting intermediate data only after every key-value pair have been processed, emit partial results after processing every *n* key-value pairs.

```
for x in mylist:
    item = x.split(" ")
    print('%s\t%s' % (item[0],item[1]))
```

Local Aggregation - Algorithm Correctness

Given a large user log from a popular website, each record consists of:

- a key -> user ids
- a value -> number of click a day

A user may have many records in the log

Local Aggregation - Algorithm Correctness

Given a large user log from a popular website, each record consists of:

- a key -> user ids
- a value -> number of click a day

A user may have many records in the log

….
<John, 1>
…
<John, 2>
…
<John, 3>
…
<John, 4>
…
<John, 5>
…

Task:
compute the mean value per user

John
(1+2+3+4+5)/5=3

First Solution

mapper.py

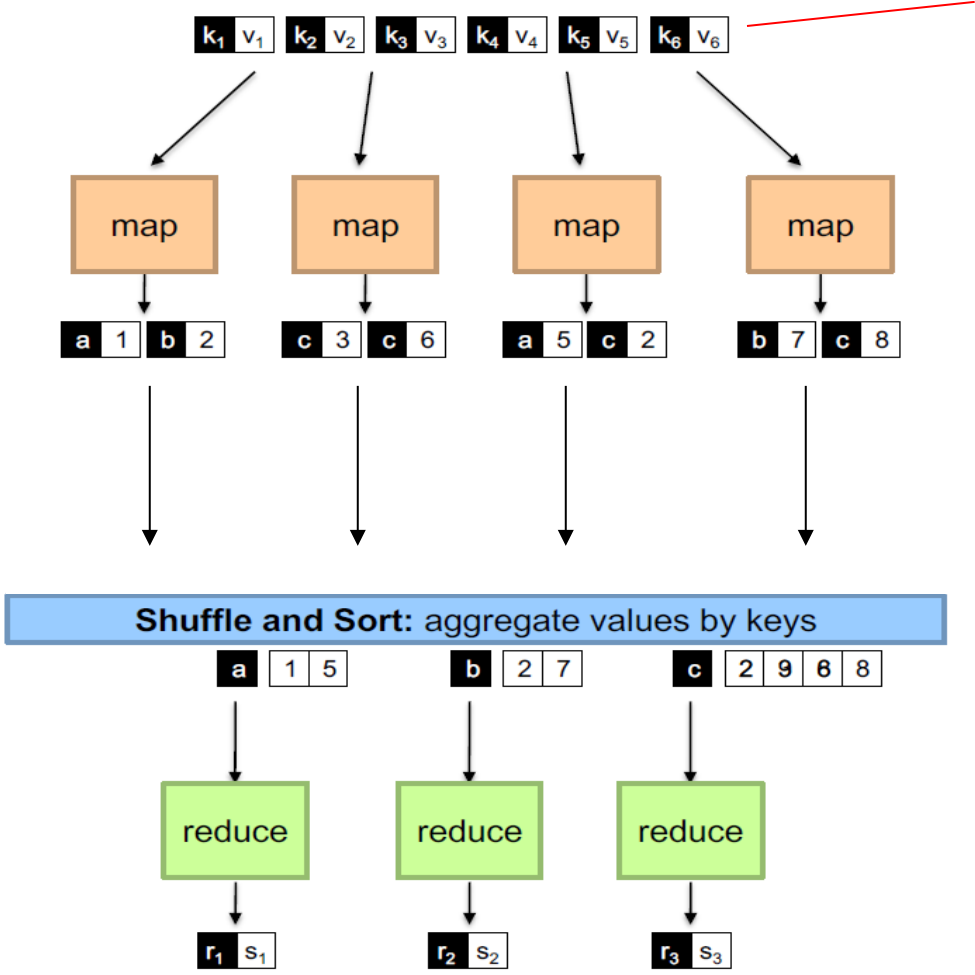
```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
  line = line.strip()
  words = line.split(" ")
  for word in words:
    print('%s\t%s' % (word,1))
```

reducer.py

```
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word,count = line.split('\t',1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word
if current_word == word:
    print('%s\t%s' % (current_word,current_count))
```

First Solution
(map and reduce only)



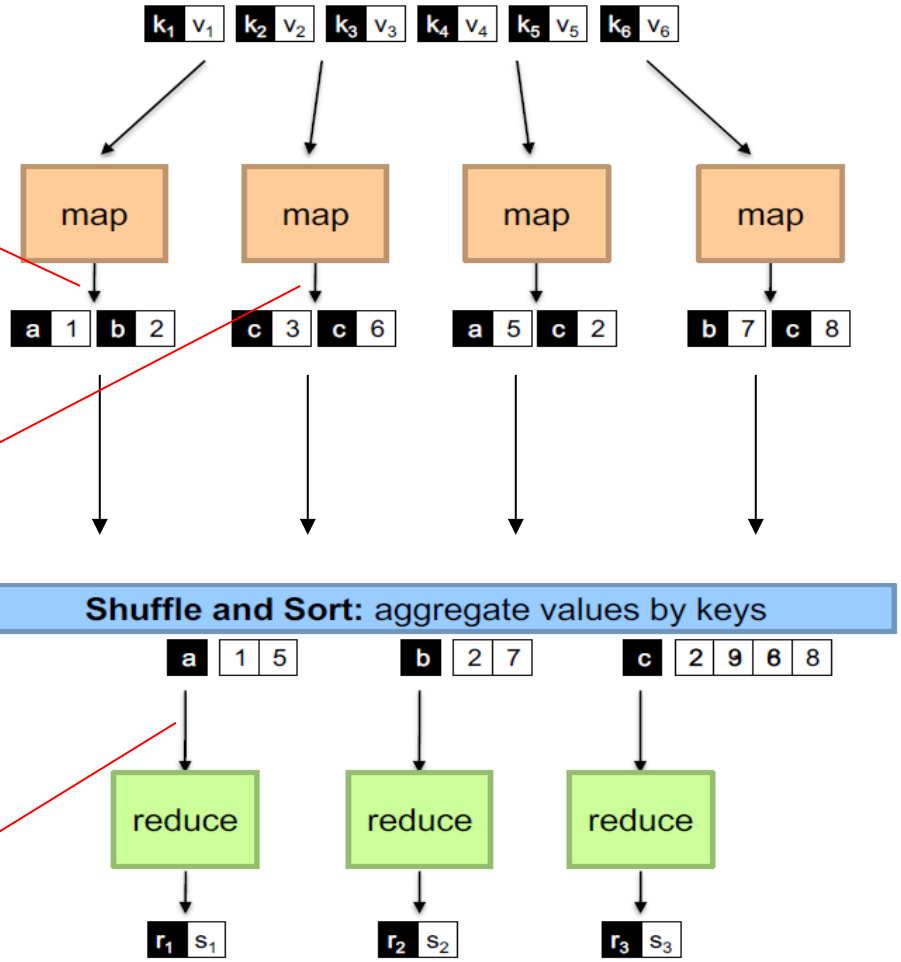
….
<John, 1>
…
<John, 2>
…
<John, 3>
…
<John, 4>
…
<John, 5>
…

First Solution
(map and reduce only)

….
<John, 1>
<John, 2>
….

….
<John, 3>
<John, 4>
<John, 5>
….

….
<John, 1>
<John, 2>
<John, 3>
<John, 4>
<John, 5>



….
<John, 1>
…
<John, 2>
…
<John, 3>
…
<John, 4>
…
<John, 5>
…

Mean(1,2,3,4,5) = 15/5 =3



Second Solution
(First Solution + Combiner)

mapper.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
  line = line.strip()
  words = line.split(" ")
  for word in words:
    print('%s\t%s' % (word,1))
```

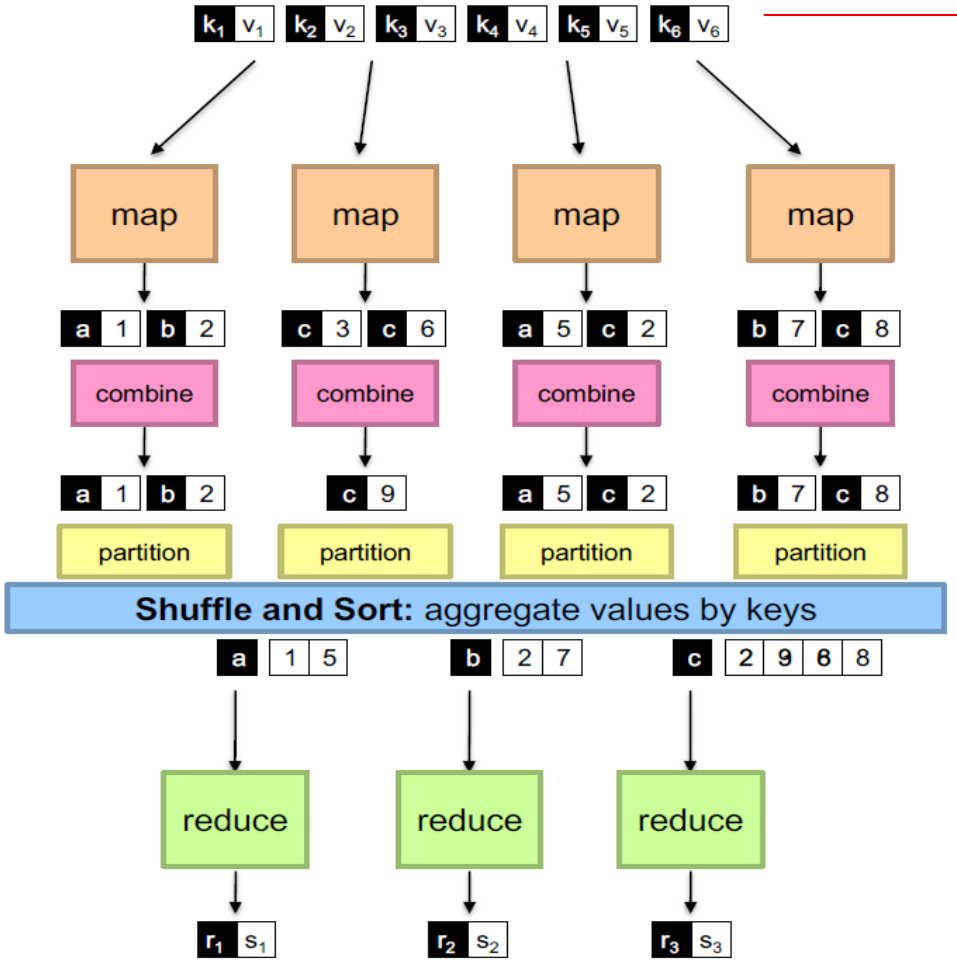
combiner.py

```
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word,count = line.split('\t',1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word
if current_word == word:
    print('%s\t%s' % (current_word,current_count))
```

reducer.py

```
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word,count = line.split('\t',1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word
if current_word == word:
    print('%s\t%s' % (current_word,current_count))
```

Second Solution
(First Solution + Combiner)



….
<John, 1>
…
<John, 2>
…
<John, 3>
…
<John, 4>
…
<John, 5>
…

Second Solution
(First Solution + Combiner)

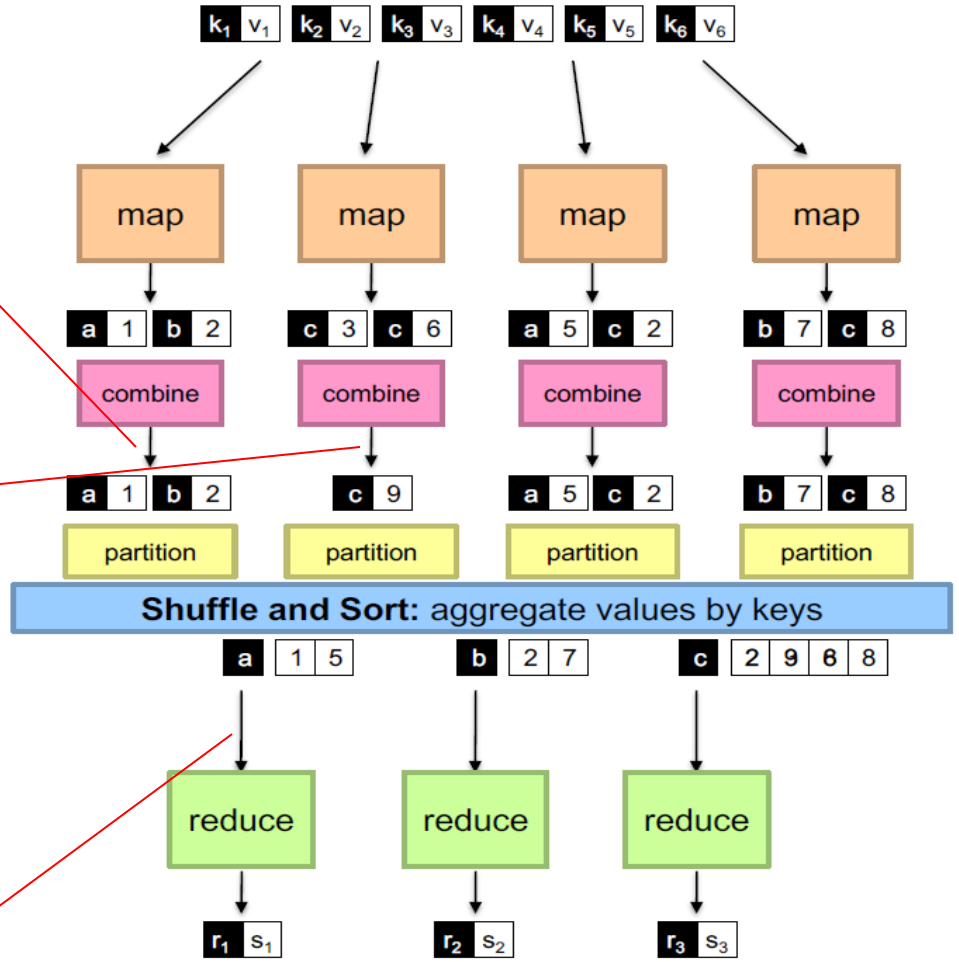
<John, 1.5>

….
~~<John, 1>~~
~~<John, 2>~~
….

….
~~<John, 3>~~
~~<John, 4>~~
~~<John, 5>~~
….

<John, 4>

<John, 1.5>
<John, 4>



….
<John, 1>
…
<John, 2>
…
<John, 3>
…
<John, 4>
…
<John, 5>
…

Mean(1.5,4) = 5.5/2 =2.75



Second Solution – incorrect

Mean is not associative and commutative

$$\textsc{Mean}(1,2,3,4,5) \neq \textsc{Mean}(\textsc{Mean}(1,2),\textsc{Mean}(3,4,5))$$

Third Solution (First Solution + Combiner +)

mapper.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
  line = line.strip()
  words = line.split(" ")
  for word in words:
    print('%s\t%s' % (word,1))
```

combiner.py

```
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word,count = line.split('\t',1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word
if current_word == word:
    print('%s\t%s' % (current_word,current_count))
```

reducer.py

```
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word,count = line.split('\t',1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word
if current_word == word:
    print('%s\t%s' % (current_word,current_count))
```

Third Solution
(First Solution + Combiner +)

mapper.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
  line = line.strip()
  words = line.split(" ")
  for word in words:
    print('%s\t%s' % (word,1))
```

combiner.py

```
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word,count = line.split('\t',1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word
if current_word == word:
    print('%s\t%s' % (current_word,current_count))
```

define a new variable num = 0

num = num + 1

num = 1

Also print

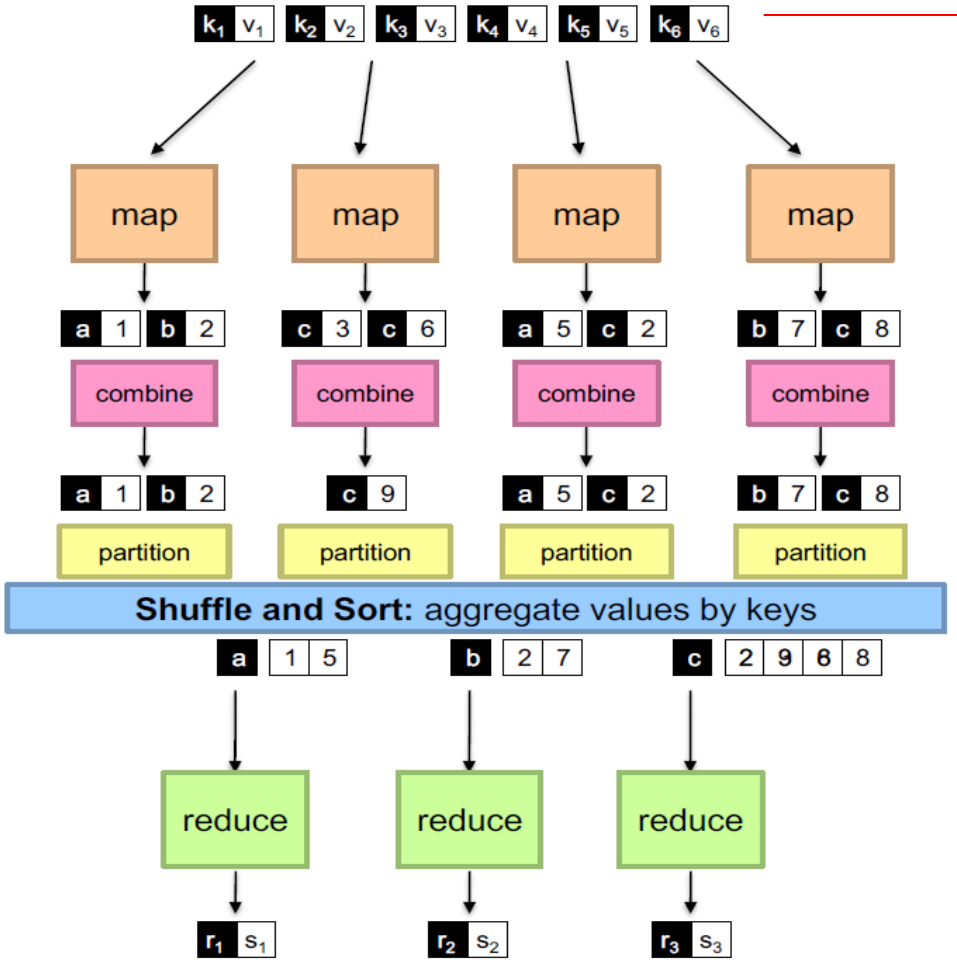
reducer.py

```
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word,count = line.split('\t',1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word
if current_word == word:
    print('%s\t%s' % (current_word,current_count))
```

Read in three items each input

Also print num

Third Solution
(First Solution + Combiner +)



….
<John, 1>
…
<John, 2>
…
<John, 3>
…
<John, 4>
…
<John, 5>
…

Third Solution
(First Solution + Combiner +)

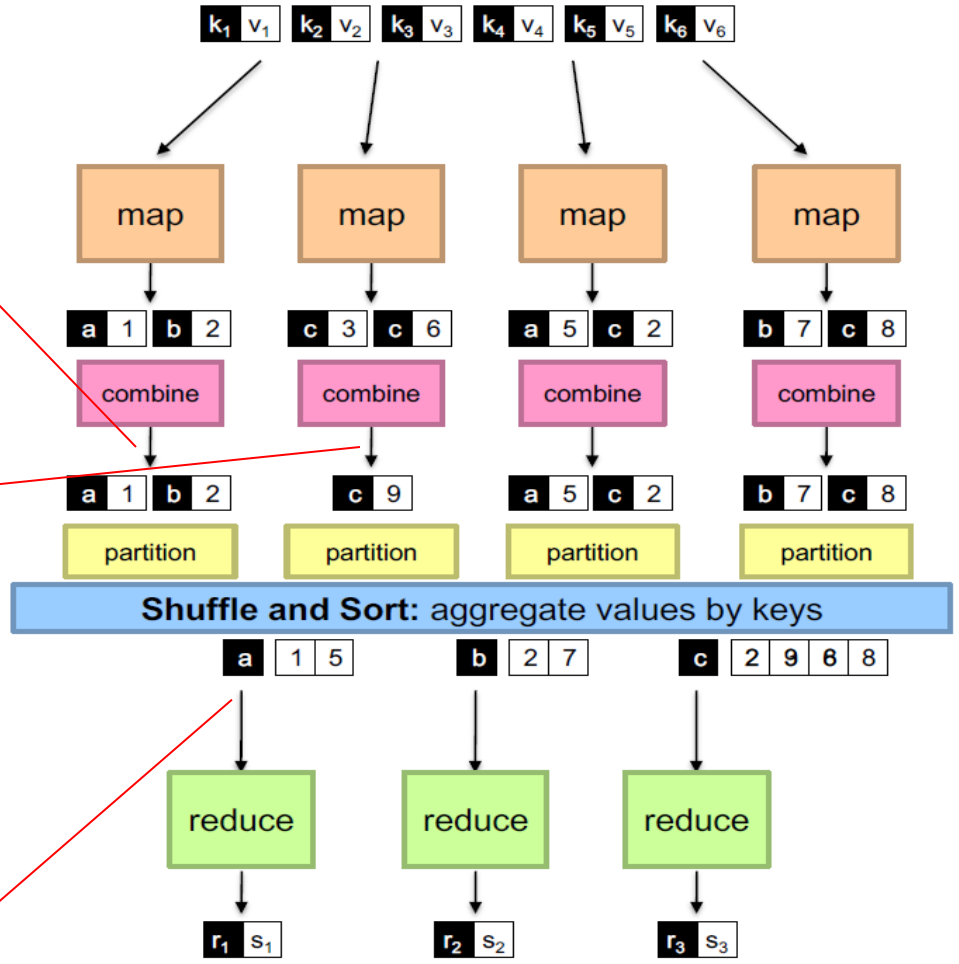
<John, 3, 2>

….
~~<John, 1>~~
~~<John, 2>~~
….

….
~~<John, 3>~~
~~<John, 4>~~
~~<John, 5>~~
….

<John, 12, 3>

<John, 3, 2>
<John, 12, 3>



….
<John, 1>
…
<John, 2>
…
<John, 3>
…
<John, 4>
…
<John, 5>
…

(3+12)/(2+3) = 15/5 = 3



Third Solution
(First Solution + Combiner +)

mapper.py

input – line by line

output – <key, value>

combiner.py

input - <key, value>

output -- <key, value, num>

reducer.py

input -- <key, value, num>

output -- <key, mean>

Third Solution
(First Solution + Combiner +)

mapper.py
- input – line by line
- output – <key, value>

combiner.py
- input - <key, value>
- output -- <key, value, num>

reducer.py
- input -- <key, value, num>
- output -- <key, mean>

Hadoop makes no guarantees on how many times the combiner is called, it could be 0, 1, or multiple times

What if combiner is called 0 times?

Third Solution
(First Solution + Combiner +)

mapper.py

input – line by line

output – <key, value>

Output of Mapper ≠ Input of Reducer

reducer.py

input -- <key, value, num>

output -- <key, mean>

Hadoop makes no guarantees on how many time the combiner is called, it could be 0, 1, or multiple times

What if combiner is called 0 times?

Fourth Solution

mapper.py

input – line by line

output – <key, value, 1>

combiner.py

input - <key, value, 1>

output -- <key, value, num>

reducer.py

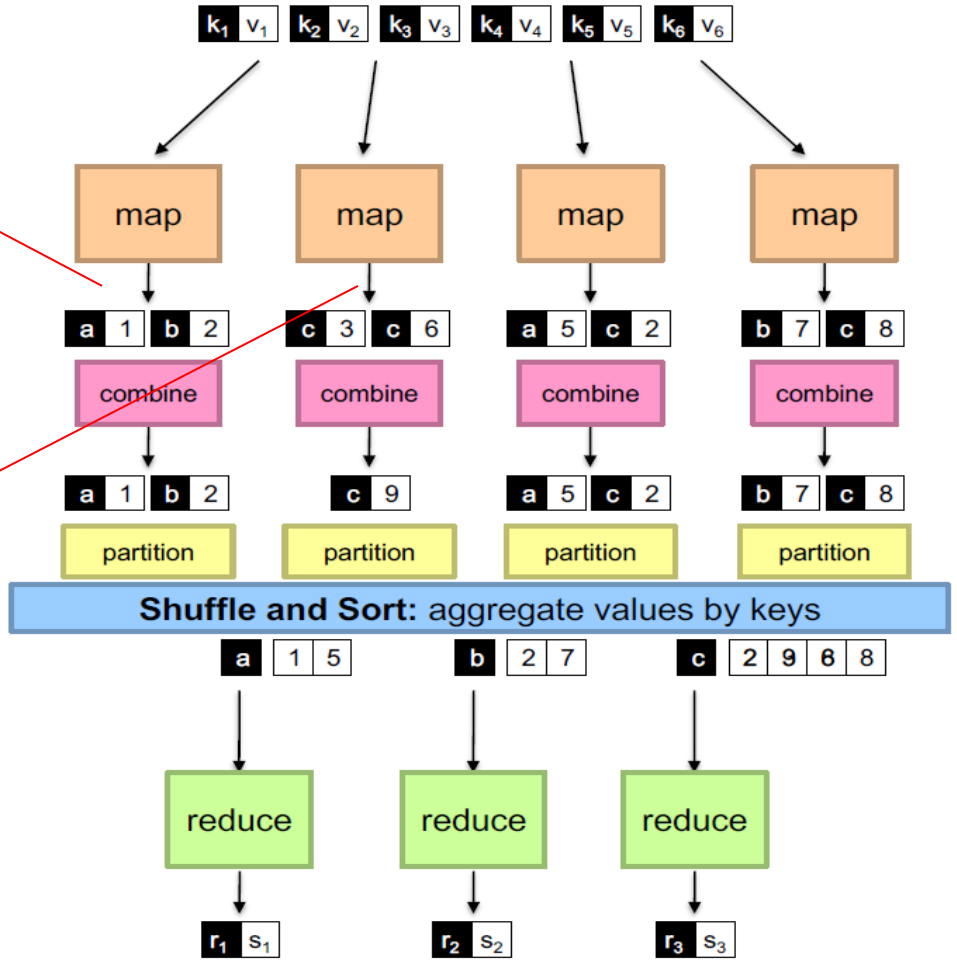
input -- <key, value, num>

output -- <key, mean>

Fourth Solution

….
<John, (1,1)>
<John, (2,1)>
….

….
<John, (3,1)>
<John, (4,1)>
<John, (5,1)>
….



….
<John, 1>
…
<John, 2>
…
<John, 3>
…
<John, 4>
…
<John, 5>
…

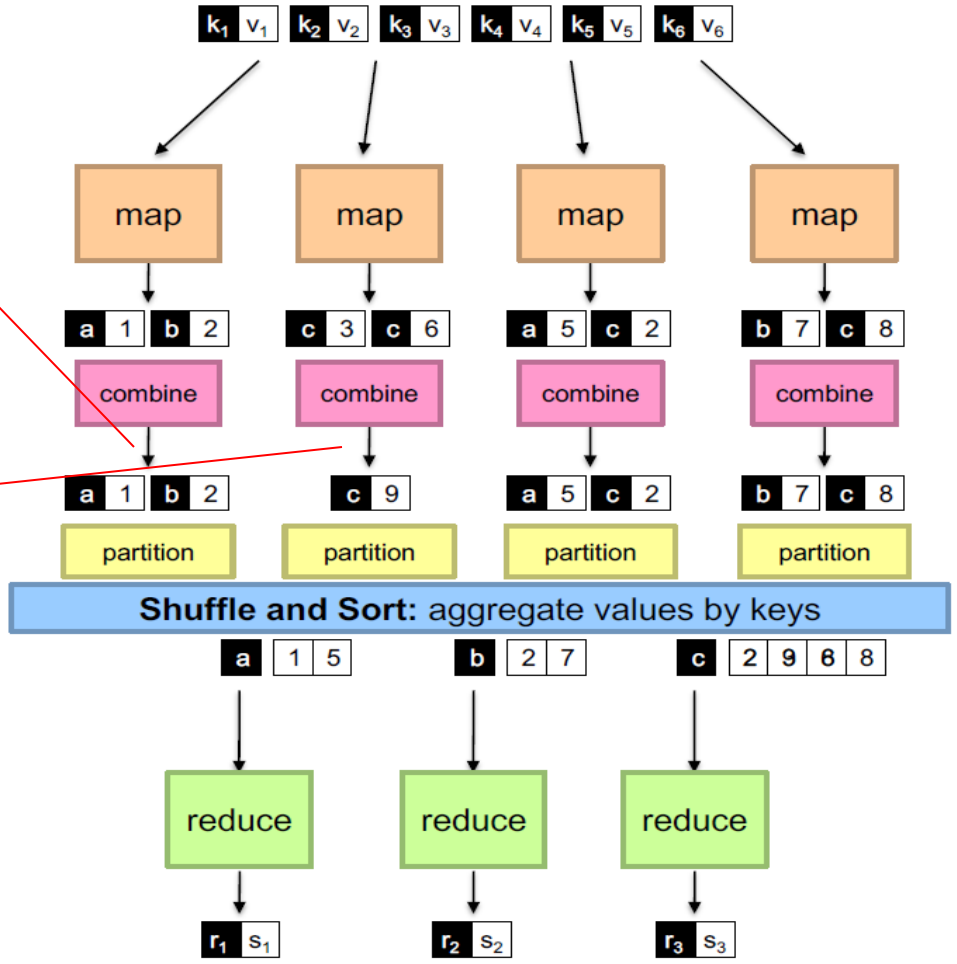
Fourth Solution

<John, 3, 2>

….
~~<John, (1,1)>~~
~~<John, (2,1)>~~
….

….
~~<John, (3,1)>~~
~~<John, (4,1)>~~
~~<John, (5,1)>~~
….

<John, 12, 3>



….
<John, 1>
…
<John, 2>
…
<John, 3>
…
<John, 4>
…
<John, 5>
…

Fourth Solution

Hadoop makes no guarantees on how many times the combiner is called, it could be 0, 1, or multiple times

mapper.py

input – line by line

output – <key, value,1>

Output of Mapper = Input of Reducer

reducer.py

input -- <key, value, num>

output -- <key, mean>

What if combiner is called 0 times?



Fifth Solution (Fourth Solution + in-mapper combining)

What is your solution?

End of Part 4

Next Week
MapReduce Algorithm Design Patterns